

DA5020 PRACTICUM 3

Members

Deepali Krishna Kashyap, Jan Mollet, Kanchan Sadashiv Deore, Nidhi Nityanand Uchil, Shamitha K V,

2025-02-16

Loading Libraries

```
library(tidyverse)
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats   1.0.0      v stringr   1.5.1
```

```
## v ggplot2    3.5.1      v tibble    3.2.1
```

```
## v lubridate  1.9.4      v tidyr     1.3.1
```

```
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
```

```
library(XML)
```

```
library(dplyr)
```

```
library(stringr)
```

```
library(tidyr)
```

```
library(ggplot2)
```

```
library(psych)
```

```
## Warning: package 'psych' was built under R version 4.4.3
```

```
##
```

```
## Attaching package: 'psych'
```

```
##
```

```
## The following objects are masked from 'package:ggplot2':
```

```
##
```

```
##      %+%, alpha
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.3
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(reshape2)
```

```
## Warning: package 'reshape2' was built under R version 4.4.3
```

```
##
## Attaching package: 'reshape2'
##
## The following object is masked from 'package:tidyr':
##
##     smiths
```

Question 1 — (20 points) + 10 optional points: DATA UNDERSTANDING

```
#Creating the data frame tripdata_df where we will store all the data coming from the ".csv" file.
tripdata_df <- read.csv("P3smaller.csv")
#We use the class() function to check if "tripdata_df" is a data frame.
head(tripdata_df)
```

```
##   VendorID lpep_pickup_datetime lpep_dropoff_datetime store_and_fwd_flag
## 1      NA 2020-02-07 09:13:00 2020-02-07 09:30:00
## 2       2 2020-02-24 15:15:36 2020-02-24 15:23:17 N
## 3       2 2020-02-27 14:28:08 2020-02-27 14:29:46 N
## 4      NA 2020-02-14 10:01:00 2020-02-14 10:16:00
## 5       2 2020-02-24 20:46:20 2020-02-24 20:55:48 N
## 6       2 2020-02-16 01:13:57 2020-02-16 01:17:57 N
##   RatecodeID PULocationID DOLocationID passenger_count trip_distance
## 1          NA          210           55             NA          4.28
## 2           1          244          244             1          1.04
## 3           1           7           7             1          0.40
## 4          NA         185           78            NA          2.54
## 5           1           41          152             1          2.10
## 6           1           95           95             1          0.68
##   fare_amount extra mta_tax tip_amount tolls_amount ehail_fee
## 1      13.34  2.75   0.0   0.00         0         NA
## 2       6.50  0.00   0.5   0.00         0         NA
## 3       3.50  0.00   0.5   0.00         0         NA
## 4      23.43  2.75   0.0   0.00         0         NA
## 5       9.50  0.50   0.5   2.16         0         NA
## 5       5.00  0.50   0.5   1.26         0         NA
##   improvement_surcharge total_amount payment_type trip_type
## 1              0.3         16.39         NA         NA
## 2              0.3          7.30          2          1
## 3              0.3          4.30          2          1
## 4              0.3         26.48         NA         NA
```

```
## 5          0.3      12.96          1          1
## 6          0.3       7.56          1          1
## congestion_surcharge
## 1          NA
## 2          0
## 3          0
## 4          NA
## 5          0
## 6          0
```

The function summary() returns the statistics values of the data frame as for example the minimum of
summary(tripdata_df)

```
## VendorID      lpep_pickup_datetime lpep_dropoff_datetime store_and_fwd_flag
## Min.      :1.000      Length:3986          Length:3986          Length:3986
## 1st Qu.:2.000      Class :character    Class :character    Class :character
## Median :2.000      Mode  :character    Mode  :character    Mode  :character
## Mean      :1.825
## 3rd Qu.:2.000
## Max.      :2.000
## NA's      :802
## RatecodeID      PULocationID      DOLocationID      passenger_count
## Min.      :1.000      Min.      : 3.0      Min.      : 3.0      Min.      :0.000
## 1st Qu.:1.000      1st Qu.: 51.0      1st Qu.: 61.0      1st Qu.:1.000
## Median :1.000      Median : 82.0      Median :129.0      Median :1.000
## Mean      :1.108      Mean      :106.5      Mean      :128.1      Mean      :1.315
## 3rd Qu.:1.000      3rd Qu.:166.0      3rd Qu.:189.0      3rd Qu.:1.000
## Max.      :5.000      Max.      :265.0      Max.      :265.0      Max.      :6.000
## NA's      :802
## trip_distance      fare_amount      extra      mta_tax
## Min.      : 0.000      Min.      :-27.47      Min.      :-1.0000      Min.      :-0.5000
## 1st Qu.: 1.040      1st Qu.: 7.00      1st Qu.: 0.0000      1st Qu.: 0.5000
## Median : 1.970      Median : 11.20      Median : 0.5000      Median : 0.5000
## Mean      : 3.366      Mean      : 15.50      Mean      : 0.8889      Mean      : 0.4009
## 3rd Qu.: 4.010      3rd Qu.: 20.32      3rd Qu.: 1.0000      3rd Qu.: 0.5000
## Max.      :40.270      Max.      :185.00      Max.      : 8.2500      Max.      : 0.5000
##
## tip_amount      tolls_amount      ehail_fee      improvement_surcharge
## Min.      : 0.0000      Min.      : 0.0000      Mode:logical      Min.      :-0.3000
## 1st Qu.: 0.0000      1st Qu.: 0.0000      NA's:3986          1st Qu.: 0.3000
## Median : 0.0000      Median : 0.0000
## Mean      : 0.9868      Mean      : 0.2695
## 3rd Qu.: 1.6600      3rd Qu.: 0.0000
## Max.      :50.0000      Max.      :13.7500
##
## total_amount      payment_type      trip_type      congestion_surcharge
## Min.      :-24.42      Min.      :1.000      Min.      :1.000      Min.      :0.000
## 1st Qu.: 8.80      1st Qu.:1.000      1st Qu.:1.000      1st Qu.:0.000
## Median : 14.16      Median :1.000      Median :1.000      Median :0.000
## Mean      : 18.67      Mean      :1.466      Mean      :1.025      Mean      :0.449
## 3rd Qu.: 24.40      3rd Qu.:2.000      3rd Qu.:1.000      3rd Qu.:0.000
## Max.      :199.10      Max.      :4.000      Max.      :2.000      Max.      :2.750
##
## NA's      :802      NA's      :802      NA's      :802
```

```
glimpse(tripdata_df)
```

```
## Rows: 3,986
## Columns: 20
## $ VendorID           <int> NA, 2, 2, NA, 2, 2, 2, 2, 2, 2, NA, 1, 2, 2, NA, ~
## $ lpep_pickup_datetime <chr> "2020-02-07 09:13:00", "2020-02-24 15:15:36", "2~
## $ lpep_dropoff_datetime <chr> "2020-02-07 09:30:00", "2020-02-24 15:23:17", "2~
## $ store_and_fwd_flag   <chr> "", "N", "N", "", "N", "N", "N", "N", "N", "N", ~
## $ RatecodeID          <int> NA, 1, 1, NA, 1, 1, 1, 1, 1, 1, NA, 1, 1, 1, NA, ~
## $ PULocationID        <int> 210, 244, 7, 185, 41, 95, 210, 74, 255, 41, 56, ~
## $ DOLocationID        <int> 55, 244, 7, 78, 152, 95, 29, 75, 49, 142, 76, 22~
## $ passenger_count     <int> NA, 1, 1, NA, 1, 1, 1, 1, 1, 1, NA, 1, 1, 1, NA, ~
## $ trip_distance       <dbl> 4.28, 1.04, 0.40, 2.54, 2.10, 0.68, 0.98, 0.39, ~
## $ fare_amount         <dbl> 13.34, 6.50, 3.50, 23.43, 9.50, 5.00, 6.00, 3.50~
## $ extra               <dbl> 2.75, 0.00, 0.00, 2.75, 0.50, 0.50, 0.00, 0.00, ~
## $ mta_tax             <dbl> 0.0, 0.5, 0.5, 0.0, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5~
## $ tip_amount          <dbl> 0.00, 0.00, 0.00, 0.00, 2.16, 1.26, 8.00, 0.86, ~
## $ tolls_amount        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ ehaul_fee           <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ improvement_surcharge <dbl> 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3~
## $ total_amount        <dbl> 16.39, 7.30, 4.30, 26.48, 12.96, 7.56, 14.80, 5.~
## $ payment_type        <int> NA, 2, 2, NA, 1, 1, 1, 1, 1, 1, NA, 2, 2, 1, NA, ~
## $ trip_type           <int> NA, 1, 1, NA, 1, 1, 1, 1, 1, 1, NA, 1, 1, 1, NA, ~
## $ congestion_surcharge <dbl> NA, 0.00, 0.00, NA, 0.00, 0.00, 0.00, 0.00, 0.00~
```

```
# Looking at the levels of each of the variables
```

```
payment_levels <- tripdata_df %>%
  pull(payment_type) %>%
  unique()
```

```
payment_levels
```

```
## [1] NA 2 1 3 4
```

```
RateCODE_levels <- tripdata_df %>%
  pull(RatecodeID) %>%
  unique()
```

```
RateCODE_levels
```

```
## [1] NA 1 5 4 2 3
```

```
PULocationID_levels <- tripdata_df %>%
  pull(PULocationID) %>%
  unique()
```

```
Locations <- tripdata_df %>%
  count(PULocationID)
```

```
head(Locations)
```

```
##  PULocationID  n
## 1             3  3
## 2             7 149
## 3             8  1
## 4             9  4
## 5            10  3
## 6            11  2
```

In order to speed things up and don't have to deal with a large data set that would be difficult to process for the KNNW regression model, We have decided to use the smaller data set. First of all, we have stored the data into the `tripdata_df` (data frame) to be able to use some functions and understand the data (1st and 2nd phase in the Crisp-DM). The `tripdata_df` has 3986 rows (observations) and 20 columns (features/variables). Within the data frame we have different types of data such as integers and characters.

Let's first do a summary of each of the variables to understand what they mean and their data type. Will we have to change the data type of some of the variables?

VendorID: Code for the provider:

1 = Creative Mobile Technologies, LLC

2 = Curb Mobility, LLC

6 = Myle Technologies Inc

lpep_pickup_datetime: Date and time when the meter was engaged.

lpep_dropoff_datetime: Date and time when the meter was disengaged.

store_and_fwd_flag: Indicates if the trip was stored before forwarding due to no server connection.

RatecodeID: Final rate code for the trip:

1 = Standard

2 = JFK

3 = Newark

4 = Nassau or Westchester

5 = Negotiated fare

6 = Group ride

99 = Null/unknown

PULocationID: Pickup TLC Taxi Zone.

DOLocationID: Drop-off TLC Taxi Zone.

passenger_count: Number of passengers.

trip_distance: Trip distance in miles.

fare_amount: Meter-calculated time-and-distance fare.

extra: Additional charges and surcharges.

mta_tax: Automatic tax based on the rate in use.

tip_amount: Tip amount (credit card tips only; cash tips excluded).

tolls_amount: Total tolls paid during the trip.

improvement_surcharge: Extra charge added at the flag drop since 2015.

total_amount: Total charged (excluding cash tips).

payment_type: How the passenger paid:

0 = Flex Fare

1 = Credit card

2 = Cash

3 = No charge

4 = Dispute

5 = Unknown

6 = Voided trip

trip_type: How the trip was initiated:

1 = Street-hail

2 = Dispatch

congestion_surcharge: NYS congestion surcharge amount.

cbd_congestion_fee: MTA congestion zone fee.

```
na_counts_df <- data.frame(
  Variable = names(tripdata_df),
  Missing_Values = colSums(is.na(tripdata_df))
)
```

We will have to take a look to the variables that contain missing values to decide what we will do, completely remove the rows? Compute some values from the mean or the median to lose less data? In step number two will handle with all of this but for now, we can have a clear visualization of the variables that we will focus on. For example, the “ehails_fee” column can be removed since it contains NA values within all the data frame.

```
# By creating this data frame we can summarize the data types of the variables, the data type that shows
data_type <- data.frame(
  Variable = names(tripdata_df),
  Type = sapply(tripdata_df, class),
  Transform_to = c("factor", "POSIXct datetime object", "POSIXct datetime object", "factor", "factor", "
  "Numeric (SAME)", "Numeric (SAME)", "Numeric (SAME)", "Numeric (SAME)", "Numeric (SAME)", "NO TRANSFO
  Reason = c( "Categorical provider ID", "Pickup timestamp", "Dropoff timestamp", "Yes/No flag", "Categ
```

)

data_type

##	Variable	Type
## VendorID	VendorID	integer
## lpep_pickup_datetime	lpep_pickup_datetime	character
## lpep_dropoff_datetime	lpep_dropoff_datetime	character
## store_and_fwd_flag	store_and_fwd_flag	character
## RatecodeID	RatecodeID	integer
## PULocationID	PULocationID	integer
## DOLocationID	DOLocationID	integer
## passenger_count	passenger_count	integer
## trip_distance	trip_distance	numeric
## fare_amount	fare_amount	numeric
## extra	extra	numeric
## mta_tax	mta_tax	numeric
## tip_amount	tip_amount	numeric
## tolls_amount	tolls_amount	numeric
## ehail_fee	ehail_fee	logical
## improvement_surcharge	improvement_surcharge	numeric
## total_amount	total_amount	numeric
## payment_type	payment_type	integer
## trip_type	trip_type	integer
## congestion_surcharge	congestion_surcharge	numeric
##	Transform_to	
## VendorID		factor
## lpep_pickup_datetime	POSIXct datetime	object
## lpep_dropoff_datetime	POSIXct datetime	object
## store_and_fwd_flag		factor
## RatecodeID		factor
## PULocationID		Integer (SAME)
## DOLocationID		Integer (SAME)
## passenger_count		Integer (SAME)
## trip_distance		Numeric (SAME)
## fare_amount		Numeric (SAME)
## extra		Numeric (SAME)
## mta_tax		Numeric (SAME)
## tip_amount		Numeric (SAME)
## tolls_amount		Numeric (SAME)
## ehail_fee	NO TRANSFORMATION	REMOVE DIRECTLY
## improvement_surcharge		Numeric (SAME)
## total_amount		Numeric (SAME)
## payment_type		factor
## trip_type		factor
## congestion_surcharge		Numeric (SAME)
##	Reason	
## VendorID	Categorical provider ID	
## lpep_pickup_datetime	Pickup timestamp	
## lpep_dropoff_datetime	Dropoff timestamp	
## store_and_fwd_flag	Yes/No flag	
## RatecodeID	Categorical rate code	
## PULocationID	Location ID (integer)	

```
## DOLocationID           Location ID (integer)
## passenger_count        Passenger count
## trip_distance           Distance in miles
## fare_amount            Fare in USD
## extra                  Additional charges
## mta_tax                 MTA tax in USD
## tip_amount             Tip in USD (target variable)
## tolls_amount           Tolls in USD
## ehail_fee              Surcharge in USD
## improvement_surcharge  Column not needed (no values)
## total_amount           Total trip charge (numeric)
## payment_type           Payment method (categorical)
## trip_type              Trip type (categorical)
## congestion_surcharge   Congestion charge
```

In other words, this will be the columns that we will have to change

```
data_type_2 <- data_type %>%
  select(Variable)%>%
  filter(Variable %in% Variable[c(1, 2, 3, 4, 5, 18, 19)])

data_type_2
```

```
##                               Variable
## VendorID                     VendorID
## lpep_pickup_datetime         lpep_pickup_datetime
## lpep_dropoff_datetime        lpep_dropoff_datetime
## store_and_fwd_flag           store_and_fwd_flag
## RatecodeID                   RatecodeID
## payment_type                 payment_type
## trip_type                    trip_type
```

In order to apply KNN, all variables need to be in a numeric data type. Nevertheless, it's a good start to perform an initial transformation of the data types, as shown in the table above, to understand what we are working with. However, at some point, we will need to convert all categorical variables (factors) into dummy variables using N – 1 level binary encoding to apply the regression model. One approach for handling date variables is to extract the year, month, day, and time (for both pickup and drop-off times) into separate columns. This will allow us to better observe distributions and interact with the data more effectively. Additionally, there may be some variables that we choose not to include in the model for predicting tip amount, as they may not be good predictors.

Variables that won't have an effect in the tip amount for sure

```
Not_good_indicator <- c("VendorID", "store_and_fwd_flag", "ehail_fee", "RateCodeID", "PULocationID", "DOLocationID", "trip_type")

Reason <- c("Won't affect the tip amount, is the provider of the code", "Record information", "It doesn't affect the tip amount")

Remove_data <- data.frame(Not_good_indicator, Reason)

Remove_data
```

```
##           Not_good_indicator
```



```

## 1          VendorID
## 2    store_and_fwd_flag
## 3          ehail_fee
## 4          RateCodeID
## 5          PULocationID
## 6          DOLocationID
## 7 lpep_pickup_datetime
## 8 lpep_dropoff_datetime
##
## 1
## 2
## 3
## 4
## 5
## 6
## 7 Even though the variable gives a lot of information, we have converted and extracted information f
## 8 Even though the variable gives a lot of information, we have converted and extracted information f

```

FEATURE ENGINEERING:

We were able to get all the Locations from the online table at the “Taxi & Limousine Commi-sion”. With that we have first of all imported the data and cleaned up (it was very messy and nested). After that, we have transformed the ID to an integer. We have done some research by borough and zone to gather the average anual salary in each of the zones. This can be a good indicator to predict the tip amount. Even though we don’t know the economic situation of each of the passengers, it can give us a good estimate. Finally, we joined the location tables to the taxis dataset and added the average salary for each borough. We used ggplot to visualize the counts of tips that are greater than 0 for each of the boroughs. These can be very important variables to predict the tip amount because each borough has a different socioeconomic status.

```

locations_df <- read_delim("taxi_zone_lookup.csv", delim = ",", col_names = FALSE)

## Rows: 266 Columns: 1
## -- Column specification -----
## Delimiter: ","
## chr (1): X1
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# Split the single column into multiple columns by commas
locations_df <- locations_df %>%
  separate(
    col = X1,
    into = c("LocationID", "Borough", "Zone", "service_zone"),
    sep = ",",
    remove = TRUE, # Remove the original column X1
  )

# Remove any extra quotes if present
locations_df <- locations_df %>%
  mutate(across(everything(), ~ gsub("'", "", .)))

```

```

# View the cleaned data
locations_df <- locations_df[-1,]
locations_df$LocationID <- as.integer(locations_df$LocationID)

# Add a new column 'Average_Household_Income' based on the borough
locations_df <- locations_df %>%
  mutate(Average_Household_Income = case_when(
    Borough == "Manhattan" ~ 175743,
    Borough == "Brooklyn" ~ 115625,
    Borough == "Queens" ~ 106667,
    Borough == "Bronx" ~ 66878,
    Borough == "Staten Island" ~ 119550,
    TRUE ~ NA_real_ # Assign NA if borough doesn't match any listed
  ))

# View the updated dataframe with the new column
locations_df <- locations_df %>%
  select(-Zone, -service_zone)

head(locations_df)

```

```

## # A tibble: 6 x 3
##   LocationID Borough      Average_Household_Income
##   <int> <chr>          <dbl>
## 1      1 EWR              NA
## 2      2 Queens          106667
## 3      3 Bronx            66878
## 4      4 Manhattan        175743
## 5      5 Staten Island    119550
## 6      6 Staten Island    119550

```

```

tripdata_df_3 <- tripdata_df %>%
  left_join(locations_df, by = c("PULocationID" = "LocationID")) # First join for pickup location

tripdata_df_4 <- tripdata_df_3 %>%
  left_join(locations_df, by = c("DOLocationID" = "LocationID"), suffix = c("_pickup", "_dropoff"))

head(tripdata_df_4)

```

```

##   VendorID lpep_pickup_datetime lpep_dropoff_datetime store_and_fwd_flag
## 1      NA 2020-02-07 09:13:00 2020-02-07 09:30:00
## 2      2 2020-02-24 15:15:36 2020-02-24 15:23:17 N
## 3      2 2020-02-27 14:28:08 2020-02-27 14:29:46 N
## 4      NA 2020-02-14 10:01:00 2020-02-14 10:16:00
## 5      2 2020-02-24 20:46:20 2020-02-24 20:55:48 N
## 6      2 2020-02-16 01:13:57 2020-02-16 01:17:57 N
##   RatecodeID PULocationID DOLocationID passenger_count trip_distance
## 1      NA      210      55      NA      4.28
## 2      1      244      244      1      1.04

```

```
## 3      1      7      7      1      0.40
## 4     NA     185     78     NA     2.54
## 5      1     41    152     1     2.10
## 6      1     95     95     1     0.68
##   fare_amount extra mta_tax tip_amount tolls_amount ehail_fee
## 1      13.34  2.75   0.0    0.00         0         NA
## 2       6.50  0.00   0.5    0.00         0         NA
## 3       3.50  0.00   0.5    0.00         0         NA
## 4      23.43  2.75   0.0    0.00         0         NA
## 5       9.50  0.50   0.5    2.16         0         NA
## 6       5.00  0.50   0.5    1.26         0         NA
##   improvement_surcharge total_amount payment_type trip_type
## 1                   0.3      16.39         NA         NA
## 2                   0.3       7.30          2          1
## 3                   0.3       4.30          2          1
## 4                   0.3      26.48         NA         NA
## 5                   0.3      12.96          1          1
## 6                   0.3       7.56          1          1
##   congestion_surcharge Borough_pickup Average_Household_Income_pickup
## 1                   NA      Brooklyn      115625
## 2                    0      Manhattan      175743
## 3                    0       Queens      106667
## 4                   NA       Bronx       66878
## 5                    0      Manhattan      175743
## 6                    0       Queens      106667
##   Borough_dropoff Average_Household_Income_dropoff
## 1      Brooklyn      115625
## 2      Manhattan      175743
## 3       Queens      106667
## 4       Bronx       66878
## 5      Manhattan      175743
## 6       Queens      106667
```

```
tripdata_df_5 <- tripdata_df_4%>%
  filter(tip_amount>0)%>%
  select(Borough_pickup, Borough_dropoff, Average_Household_Income_pickup, Average_Household_Income_dropoff)

tripdata_df_4$Borough_dropoff <- as.factor(tripdata_df_4$Borough_dropoff)
tripdata_df_4$Borough_pickup <- as.factor(tripdata_df_4$Borough_pickup)

library(ggplot2)

ggplot(tripdata_df_5, aes(x = Borough_pickup, fill = Borough_pickup)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.3) +
  scale_fill_manual(values = c(
    "Manhattan" = "lightblue",
    "Brooklyn" = "lightpink",
    "Queens" = "palegreen",
    "Bronx" = "plum",
    "Staten Island" = "lightyellow",
    "Unknown" = "gray80",
```

```

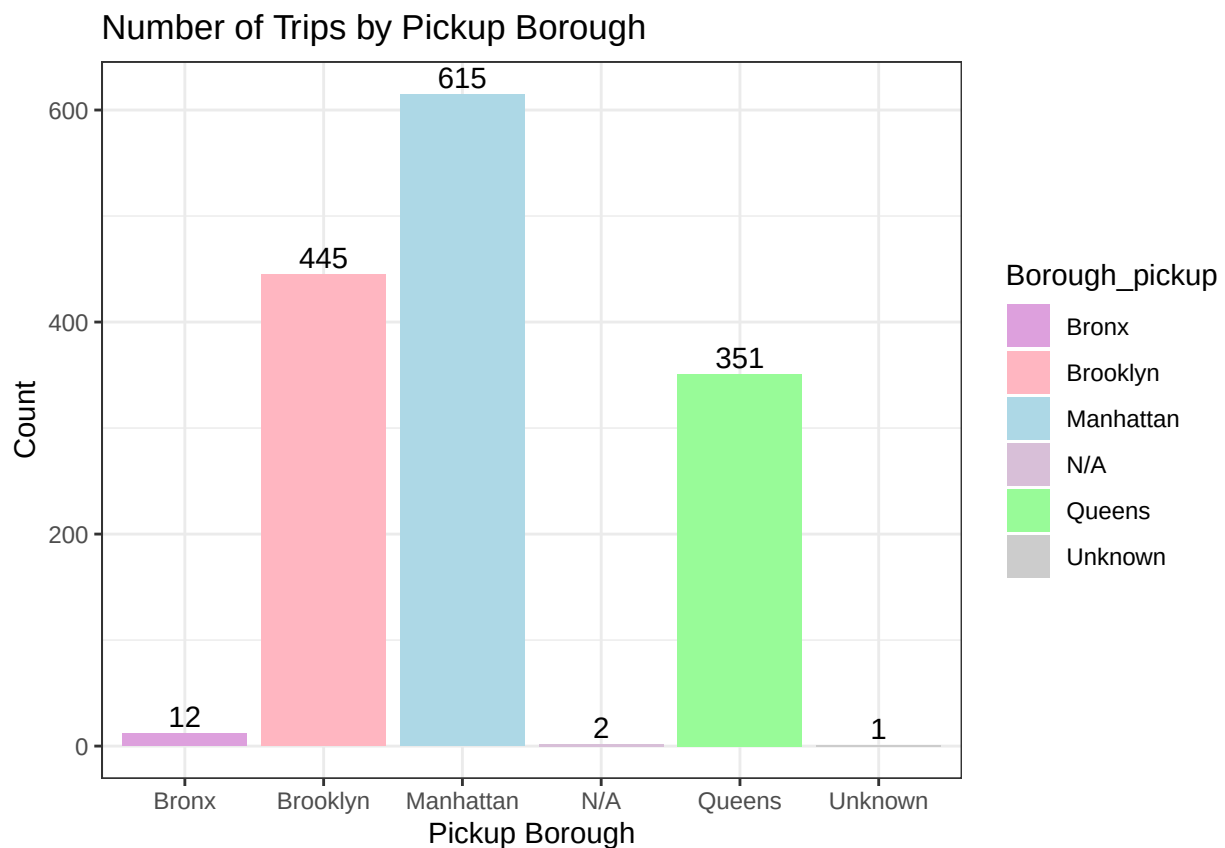
    "N/A" = "thistle"
  )) +
  labs(title = "Number of Trips by Pickup Borough",
        x = "Pickup Borough",
        y = "Count") +
  theme_bw()

```

```

## Warning: The dot-dot notation (`..count..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(count)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```



```

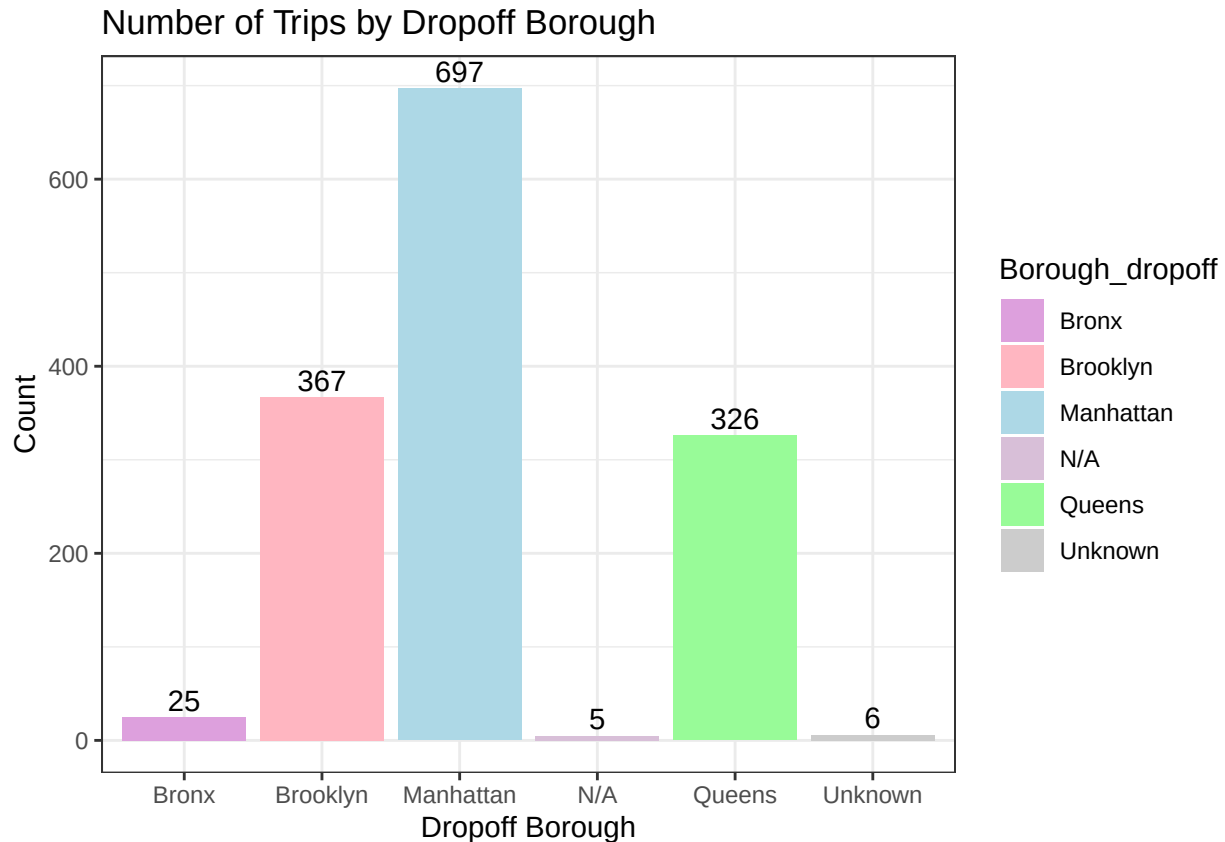
ggplot(tripdata_df_5, aes(x = Borough_dropoff, fill = Borough_dropoff)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.3) +
  scale_fill_manual(values = c(
    "Manhattan" = "lightblue",
    "Brooklyn" = "lightpink",
    "Queens" = "palegreen",
    "Bronx" = "plum",
    "Staten Island" = "lightyellow",
    "Unknown" = "gray80",
    "N/A" = "thistle"
  ))

```

```

)) +
  labs(title = "Number of Trips by Dropoff Borough",
        x = "Dropoff Borough",
        y = "Count") +
  theme_bw()

```



There are other variables that could be included to the project to predict the tip amount as for example the behaviour of the driver. In other words, a rating of the passenger explaining the behaviour of the driver (example, from 1-5 how good the driver drive or how nice was the ride in general). However, we don't have this data available so we won't be able to use it. Another good variable that we could use is peak hour/ off-peak hours. Does higher demand (peak-hour) indicate higher change to get a tip? We can also look into the day of the week. Do people tip more depending on the days of the week? Is the trip time distance a good variable?

```

tripdata_df_6 <- tripdata_df_4 %>%
  filter(!is.na(lpep_pickup_datetime) & !is.na(lpep_dropoff_datetime))

tripdata_df_6 <- tripdata_df_4 %>%
  mutate(
    lpep_pickup_datetime = as.POSIXct(lpep_pickup_datetime, format = "%Y-%m-%d %H:%M:%S"),
    lpep_dropoff_datetime = as.POSIXct(lpep_dropoff_datetime, format = "%Y-%m-%d %H:%M:%S")
  )

head(tripdata_df_6)

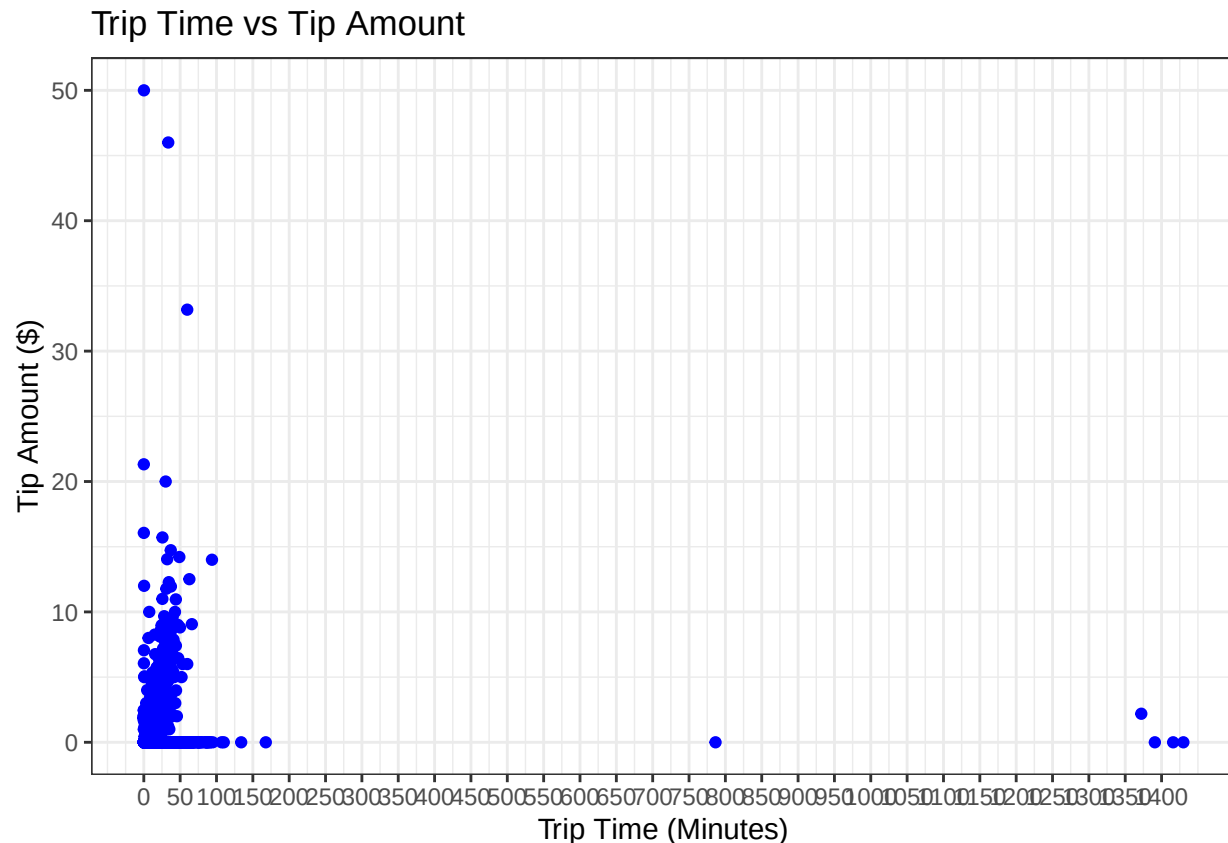
```

```
## VendorID lpep_pickup_datetime lpep_dropoff_datetime store_and_fwd_flag
```

```
## 1      NA 2020-02-07 09:13:00 2020-02-07 09:30:00
## 2      2 2020-02-24 15:15:36 2020-02-24 15:23:17      N
## 3      2 2020-02-27 14:28:08 2020-02-27 14:29:46      N
## 4      NA 2020-02-14 10:01:00 2020-02-14 10:16:00
## 5      2 2020-02-24 20:46:20 2020-02-24 20:55:48      N
## 6      2 2020-02-16 01:13:57 2020-02-16 01:17:57      N
##      RatecodeID PULocationID DOLocationID passenger_count trip_distance
## 1      NA      210      55      NA      4.28
## 2      1      244      244      1      1.04
## 3      1      7      7      1      0.40
## 4      NA      185      78      NA      2.54
## 5      1      41      152      1      2.10
## 6      1      95      95      1      0.68
##      fare_amount extra mta_tax tip_amount tolls_amount ehail_fee
## 1      13.34 2.75 0.0 0.00 0 NA
## 2      6.50 0.00 0.5 0.00 0 NA
## 3      3.50 0.00 0.5 0.00 0 NA
## 4      23.43 2.75 0.0 0.00 0 NA
## 5      9.50 0.50 0.5 2.16 0 NA
## 6      5.00 0.50 0.5 1.26 0 NA
##      improvement_surcharge total_amount payment_type trip_type
## 1      0.3 16.39 NA NA
## 2      0.3 7.30 2 1
## 3      0.3 4.30 2 1
## 4      0.3 26.48 NA NA
## 5      0.3 12.96 1 1
## 6      0.3 7.56 1 1
##      congestion_surcharge Borough_pickup Average_Household_Income_pickup
## 1      NA Brooklyn 115625
## 2      0 Manhattan 175743
## 3      0 Queens 106667
## 4      NA Bronx 66878
## 5      0 Manhattan 175743
## 6      0 Queens 106667
##      Borough_dropoff Average_Household_Income_dropoff
## 1      Brooklyn 115625
## 2      Manhattan 175743
## 3      Queens 106667
## 4      Bronx 66878
## 5      Manhattan 175743
## 6      Queens 106667
```

```
tripdata_df_7 <- tripdata_df_6 %>%
  mutate(
    trip_time_minutes = round(as.numeric(difftime(lpep_dropoff_datetime, lpep_pickup_datetime, units = "mins")))
  )

ggplot(tripdata_df_7, aes(x = trip_time_minutes, y = tip_amount)) +
  geom_point(color = "blue") + # Add points in blue
  labs(title = "Trip Time vs Tip Amount",
       x = "Trip Time (Minutes)",
       y = "Tip Amount ($)") +
  theme_bw() +
  scale_x_continuous(breaks = seq(0, max(tripdata_df_7$trip_time_minutes), by = 50))
```



```
correlation <- cor(tripdata_df_7$trip_time_minutes, tripdata_df_7$tip_amount, use = "complete.obs")

# Print the correlation result
correlation
```

```
## [1] 0.01161915
```

We have observed that the correlation is very small, meaning that it won't be a good predictor variable. However, we can still included and look at the backfitting (p-values) of the data. However, We will have to remove the outliers first

Let's try to identify the Peak hour and Off-Peak now.

```
tripdata_df_8 <- tripdata_df_7 %>%
  mutate(peak_offpeak = case_when(
    # Morning Peak: 7:00 AM - 9:00 AM (exactly on the hour)
    (hour(lpep_pickup_datetime) >= 7 & hour(lpep_pickup_datetime) < 9) ~ 1,

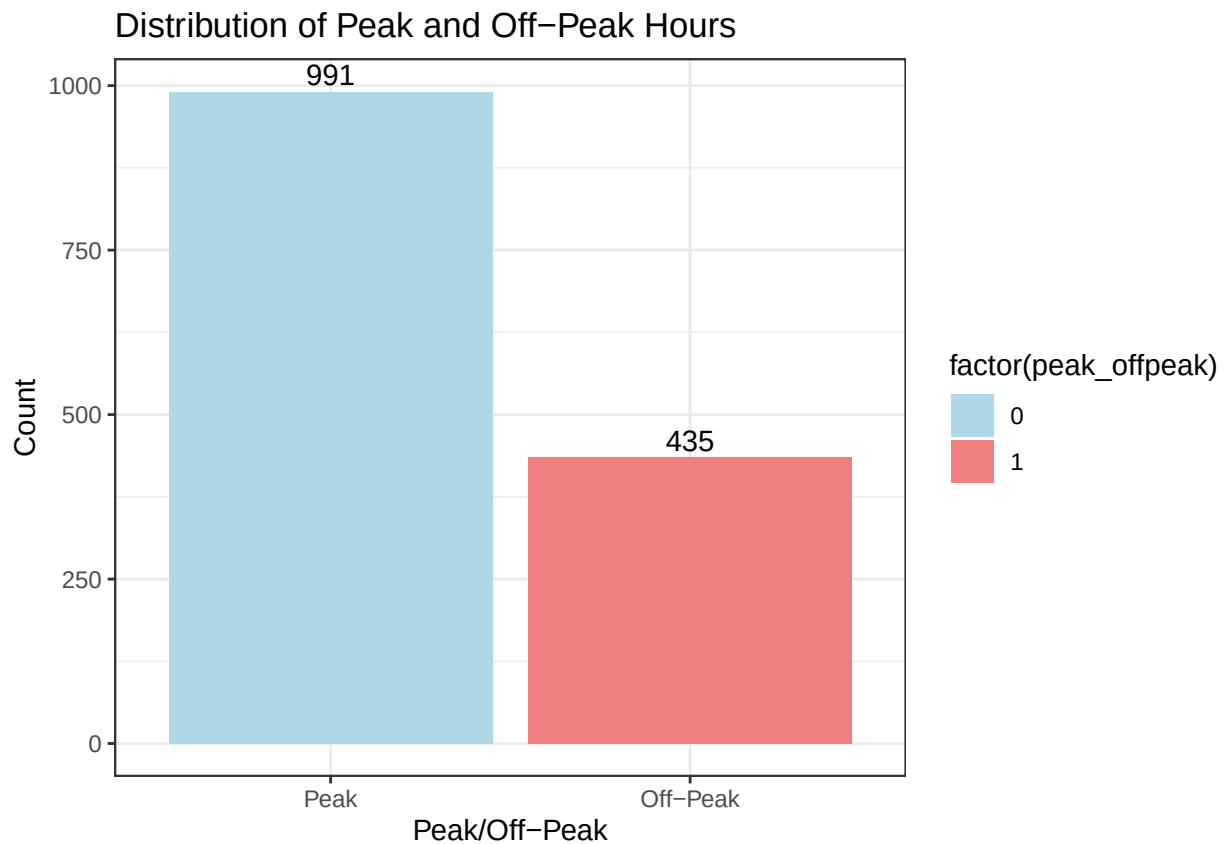
    # Evening Peak: 4:00 PM - 7:00 PM (exactly on the hour)
    (hour(lpep_pickup_datetime) >= 16 & hour(lpep_pickup_datetime) < 19) ~ 1,

    # Off-Peak: Any other time
    TRUE ~ 0
  ))
```

```
tripdata_df_9 <- tripdata_df_8%>%
  filter(tip_amount > 0)

library(ggplot2)

ggplot(tripdata_df_9, aes(x = factor(peak_offpeak), fill = factor(peak_offpeak))) +
  geom_bar() +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.3, color = "black") +
  scale_fill_manual(values = c("0" = "lightblue", "1" = "lightcoral")) +
  scale_x_discrete(labels = c("0" = "Peak", "1" = "Off-Peak")) +
  labs(
    title = "Distribution of Peak and Off-Peak Hours",
    x = "Peak/Off-Peak",
    y = "Count"
  ) +
  theme_bw()
```



That will be a good variable as well since we can see how during off-peak hours, there are more chances to get a tip than during peak hours.

```
Sys.setlocale("LC_TIME", "en_US.UTF-8") # Setting the language to english. Changing the default language
```

```
## [1] "en_US.UTF-8"
```



```

tripdata_df_10 <- tripdata_df_8%>%
  mutate(lpep_pickup_datetime = as.POSIXct(lpep_pickup_datetime, format = "%m/%d/%Y %H:%M")) %>%
  mutate(pickup_day_of_week = weekdays(lpep_pickup_datetime))

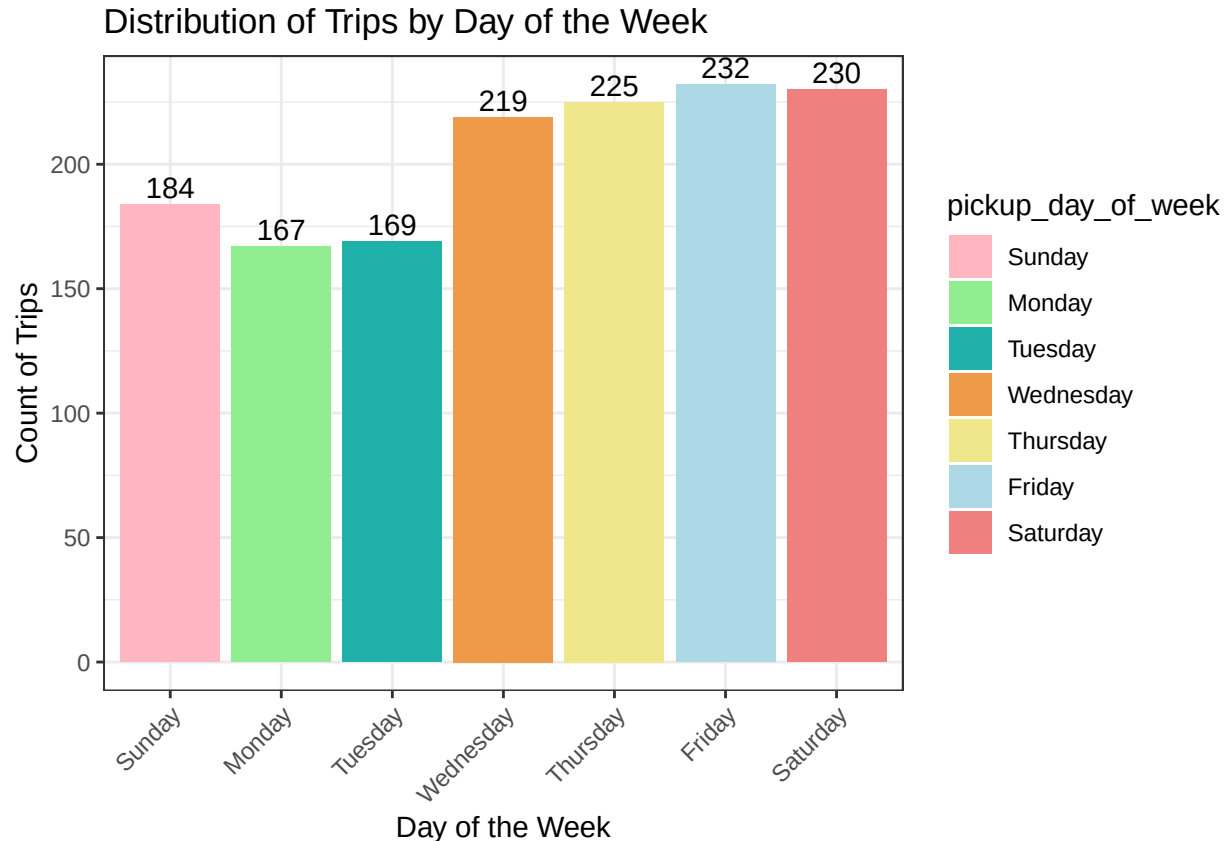
tripdata_df_11 <- tripdata_df_10%>%
  filter(tip_amount>0)

library(ggplot2)

# Convert pickup_day_of_week to a factor with ordered levels
tripdata_df_11$pickup_day_of_week <- factor(
  tripdata_df_11$pickup_day_of_week,
  levels = c("Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday")
)

ggplot(tripdata_df_11, aes(x = pickup_day_of_week, fill = pickup_day_of_week)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = ..count..), vjust = -0.3, color = "black") +
  scale_fill_manual(values = c(
    "Friday" = "lightblue",
    "Monday" = "lightgreen",
    "Saturday" = "lightcoral",
    "Sunday" = "lightpink",
    "Thursday" = "khaki",
    "Tuesday" = "lightseagreen",
    "Wednesday" = "tan2"
  )) + # Different colors for each day
  labs(
    title = "Distribution of Trips by Day of the Week",
    x = "Day of the Week",
    y = "Count of Trips"
  ) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



This variable is also very interesting as it shows the distribution of days when tips are given. The day of the week on which a pickup occurs appears to influence the likelihood of receiving a tip from a passenger.

We have now identified the variables needed for our study. Irrelevant variables that do not contribute meaningful information for predicting tips have been removed. From the original 'lpep_pickup_datetime' and 'lpep_dropoff_datetime' columns, we have derived important features such as trip duration in minutes (calculated as the difference between pickup and drop-off times), a binary indicator for whether the pickup occurred during peak hours (1) or off-peak hours (0), and the day of the week of the pickup. Once we start with our analysis, we may detect that some of the variables can also be deleted but for now we don't have enough information to evaluate that.

```
tripdata_df_clean <- tripdata_df_10%>%
  select(-VendorID, -store_and_fwd_flag, -PULocationID, -DOLocationID, -ehail_fee, -RatecodeID, -lpep_p
head(tripdata_df_clean)
```

```
##   passenger_count trip_distance fare_amount extra mta_tax tip_amount
## 1              NA          4.28      13.34  2.75    0.0    0.00
## 2              1           1.04       6.50  0.00    0.5    0.00
## 3              1           0.40       3.50  0.00    0.5    0.00
## 4              NA          2.54      23.43  2.75    0.0    0.00
## 5              1           2.10       9.50  0.50    0.5    2.16
## 6              1           0.68       5.00  0.50    0.5    1.26
##   tolls_amount improvement_surcharge total_amount payment_type trip_type
## 1            0              0.3          16.39          NA        NA
```

```
## 2      0      0.3      7.30      2      1
## 3      0      0.3      4.30      2      1
## 4      0      0.3     26.48     NA     NA
## 5      0      0.3     12.96      1      1
## 6      0      0.3      7.56      1      1
## congestion_surcharge Borough_pickup Average_Household_Income_pickup
## 1      NA      Brooklyn      115625
## 2      0      Manhattan      175743
## 3      0      Queens      106667
## 4      NA      Bronx      66878
## 5      0      Manhattan      175743
## 6      0      Queens      106667
## Borough_dropoff Average_Household_Income_dropoff trip_time_minutes
## 1      Brooklyn      115625      17.00
## 2      Manhattan      175743      7.68
## 3      Queens      106667      1.63
## 4      Bronx      66878      15.00
## 5      Manhattan      175743      9.47
## 6      Queens      106667      4.00
## peak_offpeak pickup_day_of_week
## 1      0      Friday
## 2      0      Monday
## 3      0      Thursday
## 4      0      Friday
## 5      0      Monday
## 6      0      Sunday
```

```
# Check the dimensions of the dataset
cat("DataSet Size:", nrow(tripdata_df_clean), "\n")
```

```
## DataSet Size: 3986
```

As shown in the tables above, the next steps would be handle the missing values, changing the data types and also dealing with the outliers.

Question 2 — (20 points) : DATA PREPARATION

```
# Display the structure of the data
str(tripdata_df_clean)
```

```
## 'data.frame': 3986 obs. of 19 variables:
## $ passenger_count : int NA 1 1 NA 1 1 1 1 1 1 ...
## $ trip_distance : num 4.28 1.04 0.4 2.54 2.1 0.68 0.98 0.39 2.9 2.74 ...
## $ fare_amount : num 13.3 6.5 3.5 23.4 9.5 ...
## $ extra : num 2.75 0 0 2.75 0.5 0.5 0 0 0.5 0 ...
## $ mta_tax : num 0 0.5 0.5 0 0.5 0.5 0.5 0.5 0.5 0.5 ...
## $ tip_amount : num 0 0 0 0 2.16 1.26 8 0.86 2.66 2.18 ...
## $ tolls_amount : num 0 0 0 0 0 0 0 0 0 0 ...
## $ improvement_surcharge : num 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 ...
## $ total_amount : num 16.4 7.3 4.3 26.5 13 ...
## $ payment_type : int NA 2 2 NA 1 1 1 1 1 1 ...
## $ trip_type : int NA 1 1 NA 1 1 1 1 1 1 ...
## $ congestion_surcharge : num NA 0 0 NA 0 0 0 0 0 2.75 ...
```

```
## $ Borough_pickup : Factor w/ 6 levels "Bronx","Brooklyn",...: 2 3 5 1 3 5 2 3 2 3 .
## $ Average_Household_Income_pickup : num 115625 175743 106667 66878 175743 ...
## $ Borough_dropoff : Factor w/ 6 levels "Bronx","Brooklyn",...: 2 3 5 1 3 5 2 3 2 3 .
## $ Average_Household_Income_dropoff: num 115625 175743 106667 66878 175743 ...
## $ trip_time_minutes : num 17 7.68 1.63 15 9.47 ...
## $ peak_offpeak : num 0 0 0 0 0 0 1 0 0 0 ...
## $ pickup_day_of_week : chr "Friday" "Monday" "Thursday" "Friday" ...
```

```
# Check missing values
colSums(is.na(tripdata_df_clean))
```

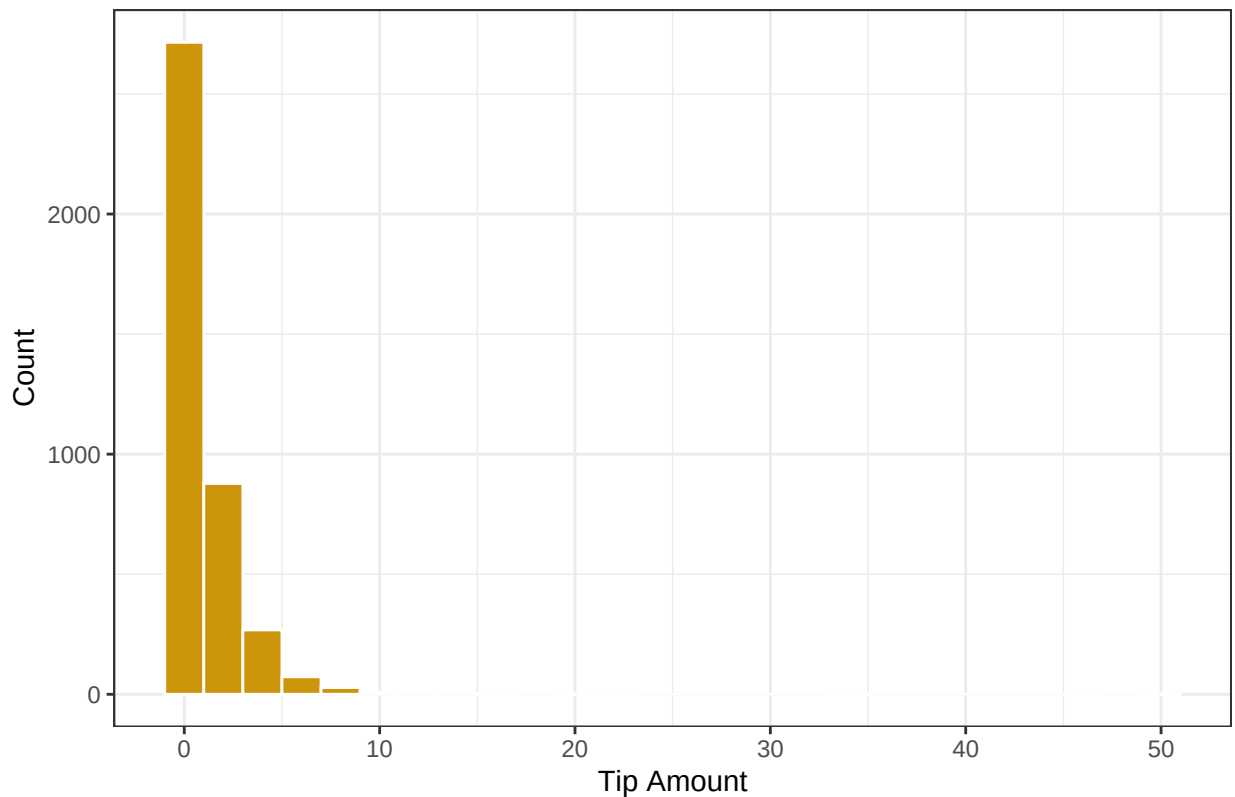
```
##           passenger_count      trip_distance
##                802                0
##           fare_amount          extra
##                0                0
##           mta_tax            tip_amount
##                0                0
##           tolls_amount      improvement_surcharge
##                0                0
##           total_amount      payment_type
##                0                802
##           trip_type      congestion_surcharge
##                802                802
##           Borough_pickup Average_Household_Income_pickup
##                0                10
##           Borough_dropoff Average_Household_Income_dropoff
##                0                34
##           trip_time_minutes      peak_offpeak
##                0                0
##           pickup_day_of_week
##                0
```

```
# Option: Remove rows where tip_amount or fare_amount is NA or 0
tripdata_df_clean <- tripdata_df_clean %>%
  filter(!is.na(tip_amount), tip_amount > 0, fare_amount > 0)
```

We begin by removing anything obviously unusable, like the `ehail_fee` column—it's completely empty and won't help us. Then, we make sure to remove any rows where the tip amount is missing, because that's the outcome we're trying to predict. If we don't know what the tip was, we can't use that row to train the model.

```
# Tip Amount Histogram - Before Cleaning
ggplot(tripdata_df, aes(x = tip_amount)) +
  geom_histogram(binwidth = 2, fill = "darkgoldenrod3", color = "white") +
  labs(title = "Tip Amount Distribution (Before Cleaning)",
       x = "Tip Amount", y = "Count") +
  theme_bw()
```

Tip Amount Distribution (Before Cleaning)



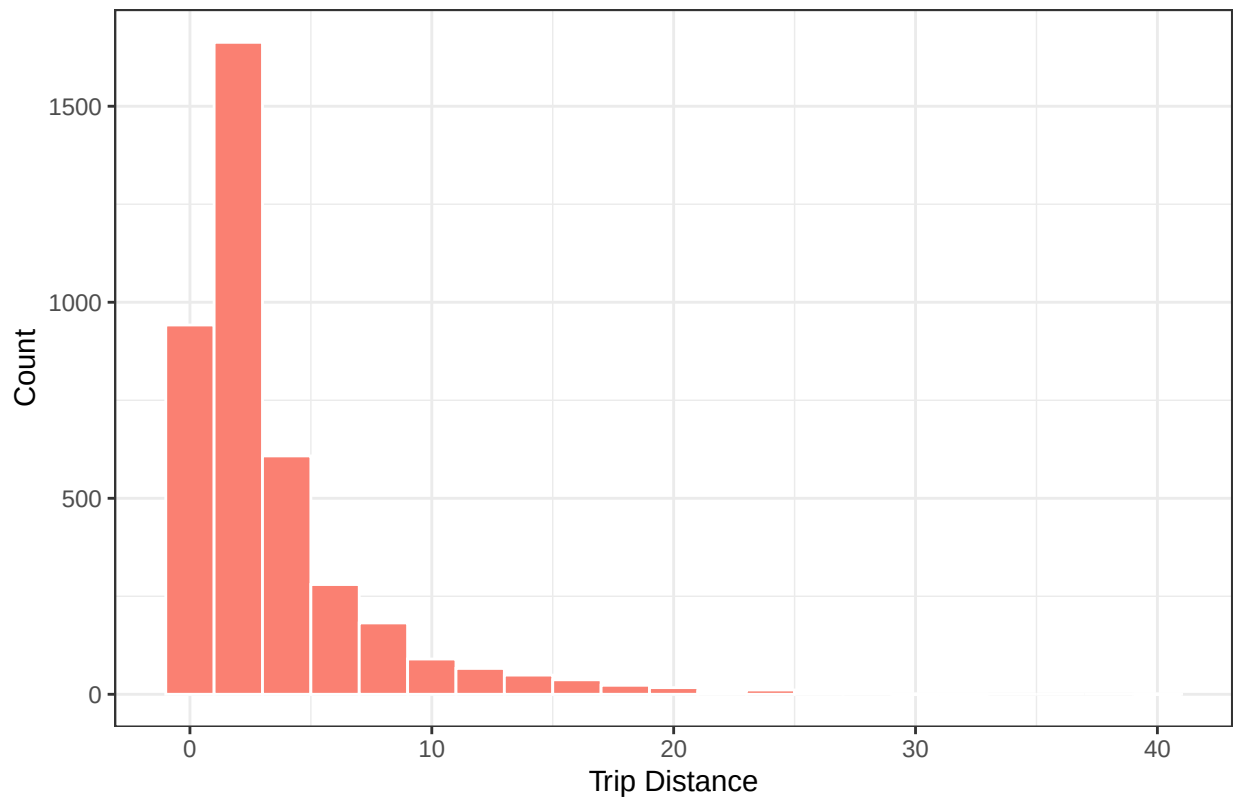
Tip Amount Histogram (Before Cleaning):

When we looked at the raw tip data, we saw tons of rides with no tips and a few with crazy high ones over \$50. It was all over the place and didn't really make sense for most normal rides. That's when we knew we had to clean it up to get a more realistic picture.

- Peak at 0: The first thing that stands out is the huge spike around \$0. This likely indicates that there are a lot of zero or missing tip amounts in the dataset—perhaps due to data entry errors or customers who didn't leave tips at all.
- Small number of larger tips: The remaining tips above \$0 are fewer and spread out. The distribution is heavily skewed toward the lower tip amounts.
- Possible outliers: There are a few larger tip amounts, but they are very rare. This suggests there might be some outliers or anomalies in the data that don't represent typical tipping behavior.

```
# Trip Distance Histogram - Before Cleaning
ggplot(tripdata_df, aes(x = trip_distance)) +
  geom_histogram(binwidth = 2, fill = "salmon", color = "white") +
  labs(title = "Trip Distance Distribution (Before Cleaning)",
       x = "Trip Distance", y = "Count") +
  theme_bw()
```

Trip Distance Distribution (Before Cleaning)



Trip Distance Histogram (Before Cleaning):

Some rides had zero distance and others were way too long—like over 50 miles, which felt off for city trips. It looked like there were some errors, maybe from GPS issues or just bad data. So, we decided to clean it up and keep only the trips that made sense.

- Peak at 0: Just like with tips, there's a massive spike at zero on the left, suggesting a large number of zero or very short trips—again, potentially due to missing data or errors.
- Longer trips are rare: As the trip distance increases, the number of occurrences decreases rapidly. Distances of 10 units or more are quite rare.
- Skewed distribution: This also shows a right-skewed distribution, where most trips are short, and longer trips are less common. The data needs some cleaning to remove the noise from these zero or extremely short trips.

```
# Remove outliers based on tip and distance thresholds
tripdata_df_clean <- tripdata_df_clean %>%
  filter(tip_amount < 50, trip_distance > 0 & trip_distance < 50)

# Remove all rows with any NA values
tripdata_df_clean <- tripdata_df_clean %>% drop_na()

colSums(is.na(tripdata_df_clean))
```

```
##                passenger_count                trip_distance
##                                0                            0
```

```
##          fare_amount          extra
##          0          0
##          mta_tax          tip_amount
##          0          0
##          tolls_amount          improvement_surcharge
##          0          0
##          total_amount          payment_type
##          0          0
##          trip_type          congestion_surcharge
##          0          0
##          Borough_pickup Average_Household_Income_pickup
##          0          0
##          Borough_dropoff Average_Household_Income_dropoff
##          0          0
##          trip_time_minutes          peak_offpeak
##          0          0
##          pickup_day_of_week
##          0
```

```
head(tripdata_df_clean)
```

```
## passenger_count trip_distance fare_amount extra mta_tax tip_amount
## 1          1          2.10          9.5  0.5  0.5  2.16
## 2          1          0.68          5.0  0.5  0.5  1.26
## 3          1          0.98          6.0  0.0  0.5  8.00
## 4          1          0.39          3.5  0.0  0.5  0.86
## 5          1          2.90         12.0  0.5  0.5  2.66
## 6          1          2.74         11.0  0.0  0.5  2.18
## tolls_amount improvement_surcharge total_amount payment_type trip_type
## 1          0          0.3          12.96          1          1
## 2          0          0.3          7.56          1          1
## 3          0          0.3         14.80          1          1
## 4          0          0.3          5.16          1          1
## 5          0          0.3         15.96          1          1
## 6          0          0.3         16.73          1          1
## congestion_surcharge Borough_pickup Average_Household_Income_pickup
## 1          0.00      Manhattan          175743
## 2          0.00      Queens          106667
## 3          0.00      Brooklyn          115625
## 4          0.00      Manhattan          175743
## 5          0.00      Brooklyn          115625
## 6          2.75      Manhattan          175743
## Borough_dropoff Average_Household_Income_dropoff trip_time_minutes
## 1      Manhattan          175743          9.47
## 2      Queens          106667          4.00
## 3      Brooklyn          115625          6.53
## 4      Manhattan          175743          1.88
## 5      Brooklyn          115625          14.43
## 6      Manhattan          175743          12.28
## peak_offpeak pickup_day_of_week
## 1          0      Monday
## 2          0      Sunday
## 3          1      Tuesday
## 4          0      Sunday
```

```
## 5          0          Thursday
## 6          0          Saturday
```

```
# Check the dimensions of the dataset
cat("DataSet Size:", nrow(tripdata_df_clean), "\n")
```

```
## DataSet Size: 1405
```

```
# Z-score normalization for continuous features
tripdata_df_clean <- tripdata_df_clean %>%
  mutate(
    fare_amount_z = scale(fare_amount),
    trip_distance_z = scale(trip_distance),
    total_amount_z = scale(total_amount)
  )
```

In this step, we standardize our numeric features using R's `scale()` function. That means we adjust each feature to have a mean of 0 and a standard deviation of 1.

```
tripdata_df_clean <- tripdata_df_clean %>%
  mutate(
    payment_type = as.factor(payment_type)
  )

# Select columns to encode
categorical_cols <- c("Borough_pickup", "Borough_dropoff", "pickup_day_of_week") # exclude payment_type
one_hot_encoded <- model.matrix(~ Borough_pickup + Borough_dropoff + pickup_day_of_week,
                                data = tripdata_df_clean)[, -1]

# Combine one-hot columns back with the original numeric dataset
tripdata_encoded <- cbind(
  tripdata_df_clean %>% select(-all_of(categorical_cols)), # remove original categorical cols
  one_hot_encoded # add encoded columns
)

str(tripdata_encoded)
```

```
## 'data.frame': 1405 obs. of 35 variables:
## $ passenger_count : int 1 1 1 1 1 1 1 1 1 1 ...
## $ trip_distance : num 2.1 0.68 0.98 0.39 2.9 2.74 2.51 3.34 2.03 0.66 ...
## $ fare_amount : num 9.5 5 6 3.5 12 11 12 13.5 7.5 5.5 ...
## $ extra : num 0.5 0.5 0 0 0.5 0 0 1 0.5 0 ...
## $ mta_tax : num 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 ...
## $ tip_amount : num 2.16 1.26 8 0.86 2.66 2.18 2.56 2.3 1.76 1.58 ...
## $ tolls_amount : num 0 0 0 0 0 0 0 0 0 0 ...
## $ improvement_surcharge : num 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 ...
## $ total_amount : num 12.96 7.56 14.8 5.16 15.96 ...
## $ payment_type : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## $ trip_type : int 1 1 1 1 1 1 1 1 1 1 ...
## $ congestion_surcharge : num 0 0 0 0 0 2.75 0 0 0 0 ...
```



```

## $ Average_Household_Income_pickup : num 175743 106667 115625 175743 115625 ...
## $ Average_Household_Income_dropoff: num 175743 106667 115625 175743 115625 ...
## $ trip_time_minutes : num 9.47 4 6.53 1.88 14.43 ...
## $ peak_offpeak : num 0 0 1 0 0 0 1 1 0 0 ...
## $ fare_amount_z : num [1:1405, 1] -0.2856 -0.8236 -0.704 -1.0029 0.0133 ...
## ..- attr(*, "scaled:center")= num 11.9
## ..- attr(*, "scaled:scale")= num 8.36
## $ trip_distance_z : num [1:1405, 1] -0.2432 -0.7792 -0.666 -0.8887 0.0588 ...
## ..- attr(*, "scaled:center")= num 2.74
## ..- attr(*, "scaled:scale")= num 2.65
## $ total_amount_z : num [1:1405, 1] -0.3375 -0.8357 -0.1678 -1.0571 -
0.0607 ...
## ..- attr(*, "scaled:center")= num 16.6
## ..- attr(*, "scaled:scale")= num 10.8
## $ Borough_pickupBrooklyn : num 0 0 1 0 1 0 0 0 1 0 ...
## $ Borough_pickupManhattan : num 1 0 0 1 0 1 1 1 0 1 ...
## $ Borough_pickupN/A : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Borough_pickupQueens : num 0 1 0 0 0 0 0 0 0 0 ...
## $ Borough_pickupUnknown : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Borough_dropoffBrooklyn : num 0 0 1 0 1 0 0 0 1 0 ...
## $ Borough_dropoffManhattan : num 1 0 0 1 0 1 1 1 0 1 ...
## $ Borough_dropoffN/A : num 0 0 0 0 0 0 0 0 0 0 ...
## $ Borough_dropoffQueens : num 0 1 0 0 0 0 0 0 0 0 ...
## $ Borough_dropoffUnknown : num 0 0 0 0 0 0 0 0 0 0 ...
## $ pickup_day_of_weekMonday : num 1 0 0 0 0 0 1 0 0 0 ...
## $ pickup_day_of_weekSaturday : num 0 0 0 0 0 1 0 0 0 0 ...
## $ pickup_day_of_weekSunday : num 0 1 0 1 0 0 0 0 0 0 ...
## $ pickup_day_of_weekThursday : num 0 0 0 0 1 0 0 1 0 0 ...
## $ pickup_day_of_weekTuesday : num 0 0 1 0 0 0 0 0 1 1 ...
## $ pickup_day_of_weekWednesday : num 0 0 0 0 0 0 0 0 0 0 ...

```

```
head(tripdata_encoded)
```

```

## passenger_count trip_distance fare_amount extra mta_tax tip_amount
## 1 1 2.10 9.5 0.5 0.5 2.16
## 2 1 0.68 5.0 0.5 0.5 1.26
## 3 1 0.98 6.0 0.0 0.5 8.00
## 4 1 0.39 3.5 0.0 0.5 0.86
## 5 1 2.90 12.0 0.5 0.5 2.66
## 6 1 2.74 11.0 0.0 0.5 2.18
## tolls_amount improvement_surcharge total_amount payment_type trip_type
## 1 0 0.3 12.96 1 1
## 2 0 0.3 7.56 1 1
## 3 0 0.3 14.80 1 1
## 4 0 0.3 5.16 1 1
## 5 0 0.3 15.96 1 1
## 6 0 0.3 16.73 1 1
## congestion_surcharge Average_Household_Income_pickup
## 1 0.00 175743
## 2 0.00 106667
## 3 0.00 115625
## 4 0.00 175743
## 5 0.00 115625
## 6 2.75 175743

```

##	Average_Household_Income_dropoff	trip_time_minutes	peak_offpeak	fare_amount_z
## 1	175743	9.47	0	-0.28560360
## 2	106667	4.00	0	-0.82358327
## 3	115625	6.53	1	-0.70403223
## 4	175743	1.88	0	-1.00290982
## 5	115625	14.43	0	0.01327399
## 6	175743	12.28	0	-0.10627704
##	trip_distance_z	total_amount_z	Borough_pickupBrooklyn	Borough_pickupManhattan
## 1	-0.243195085	-0.33751424	0	1
## 2	-0.779213506	-0.83569532	0	0
## 3	-0.665970178	-0.16776365	1	0
## 4	-0.888682057	-1.05710914	0	1
## 5	0.058787124	-0.06074697	1	0
## 6	-0.001609318	0.01028996	0	1
##	Borough_pickupN/A	Borough_pickupQueens	Borough_pickupUnknown	
## 1	0	0	0	
## 2	0	1	0	
## 3	0	0	0	
## 4	0	0	0	
## 5	0	0	0	
## 6	0	0	0	
##	Borough_dropoffBrooklyn	Borough_dropoffManhattan	Borough_dropoffN/A	
## 1	0	1	0	
## 2	0	0	0	
## 3	1	0	0	
## 4	0	1	0	
## 5	1	0	0	
## 6	0	1	0	
##	Borough_dropoffQueens	Borough_dropoffUnknown	pickup_day_of_weekMonday	
## 1	0	0	1	
## 2	1	0	0	
## 3	0	0	0	
## 4	0	0	0	
## 5	0	0	0	
## 6	0	0	0	
##	pickup_day_of_weekSaturday	pickup_day_of_weekSunday		
## 1	0	0		
## 2	0	1		
## 3	0	0		
## 4	0	1		
## 5	0	0		
## 6	1	0		
##	pickup_day_of_weekThursday	pickup_day_of_weekTuesday		
## 1	0	0		
## 2	0	0		
## 3	0	1		
## 4	0	0		
## 5	1	0		
## 6	0	0		
##	pickup_day_of_weekWednesday			
## 1	0			
## 2	0			
## 3	0			
## 4	0			

```
## 5          0
## 6          0
```

Machine learning models don't understand text like "Cash" or "Credit Card", so we convert these categories into numbers using one-hot encoding. This turns each category into its own column—if the row matches that category, we put a 1, otherwise a 0.

```
set.seed(123) # for reproducibility

# Shuffle the rows
tripdata_encoded <- tripdata_encoded[sample(nrow(tripdata_encoded), )

# 80% training, 20% testing
train_index <- floor(0.8 * nrow(tripdata_encoded))

data_train <- tripdata_encoded[1:train_index, ]
data_test  <- tripdata_encoded[(train_index + 1):nrow(tripdata_encoded), ]

# Check the dimensions of the dataset
cat("DataSet Size:", nrow(tripdata_encoded), "\n")
```

```
## DataSet Size: 1405
```

```
# Check the dimensions of the split
cat("Training Set Size:", nrow(data_train), "\n")
```

```
## Training Set Size: 1124
```

```
cat("Test Set Size:", nrow(data_test), "\n")
```

```
## Test Set Size: 281
```

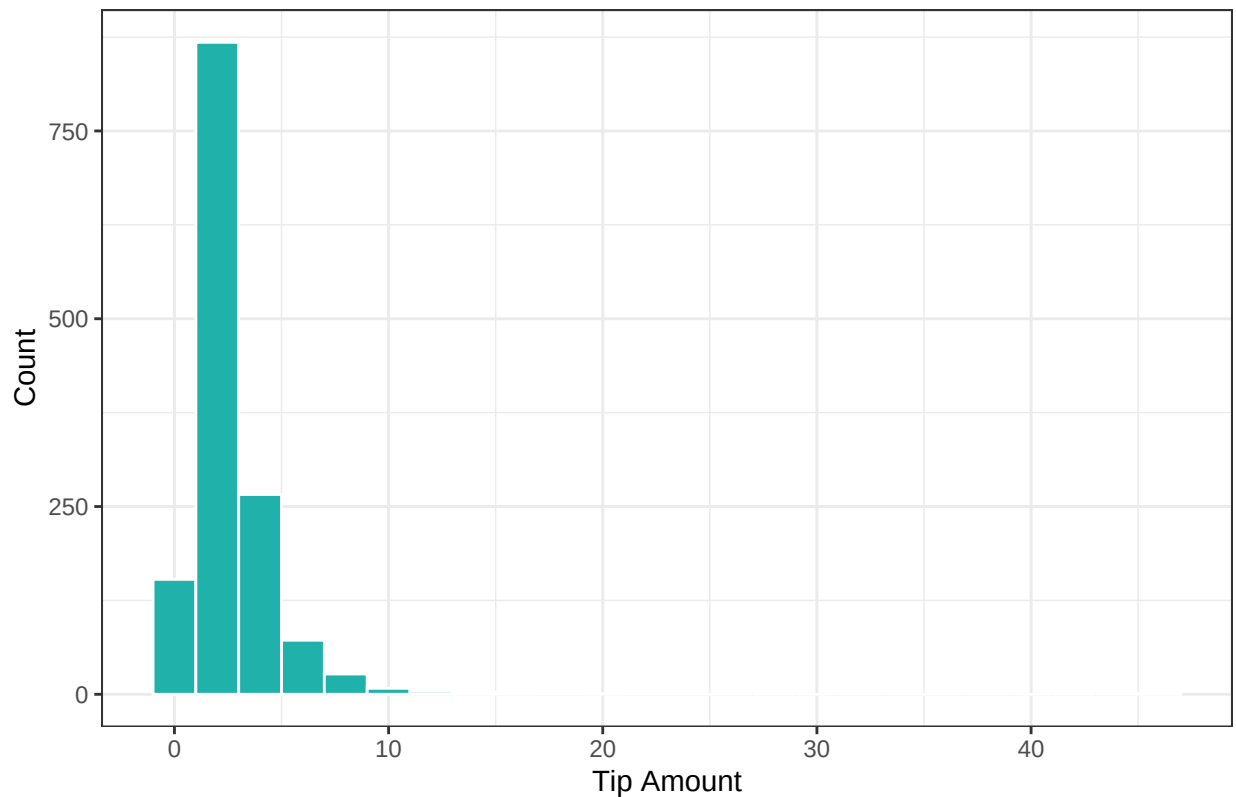
Now that we've cleaned, fixed, and encoded everything, we combine the numeric and encoded features into one big dataset `x`, and our target variable (`tip_amount`) into `y`.

Then we shuffle the data (so the order doesn't influence anything) and split it into training and test sets using an 80/20 ratio:

- Training set (80%) is what the model learns from.
- Test set (20%) is used later to see how well the model performs on new, unseen data.

```
# Histogram AFTER cleaning
ggplot(tripdata_df_clean, aes(x = tip_amount)) +
  geom_histogram(binwidth = 2, fill = "lightseagreen", color = "white") +
  labs(title = "Tip Amount Distribution (After Cleaning)",
       x = "Tip Amount", y = "Count") +
  theme_bw()
```

Tip Amount Distribution (After Cleaning)



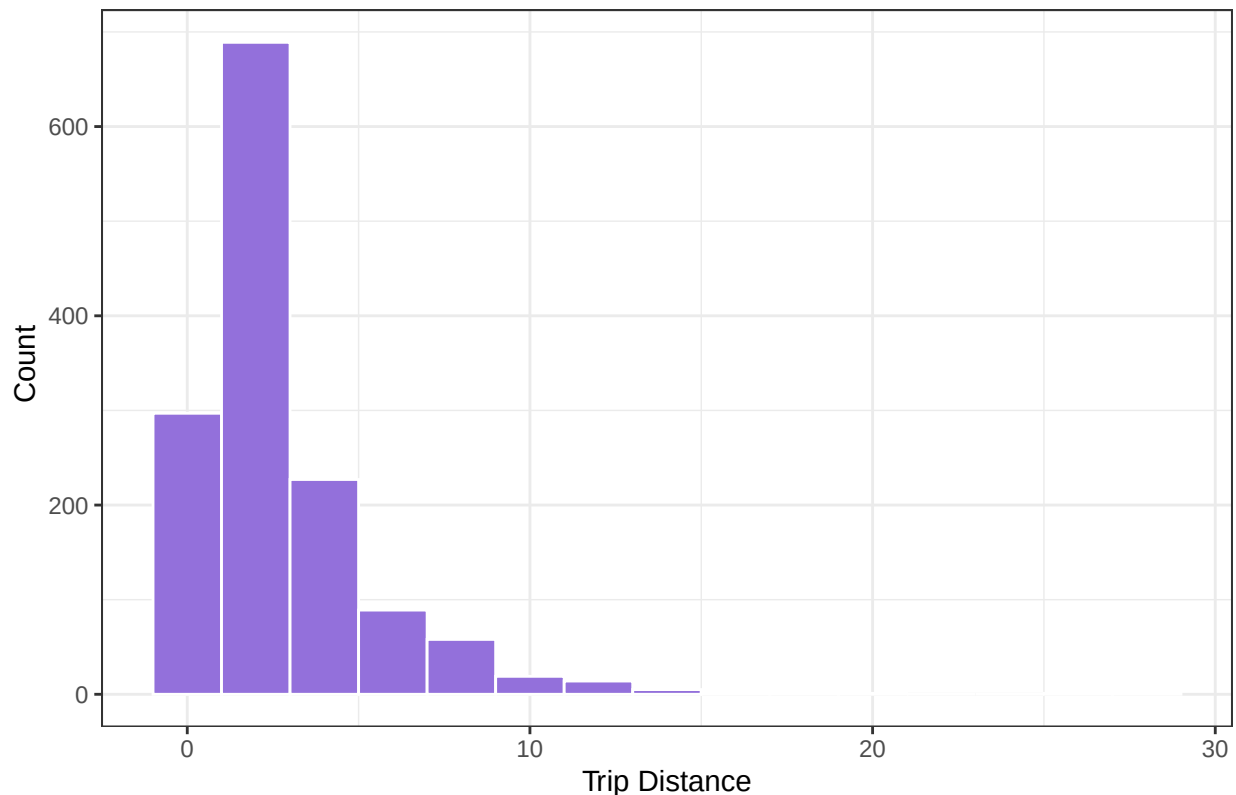
Tip Amount Histogram:(After Cleaning)

After cleaning the data by removing unusually high tips and missing values, we can see that most riders tend to tip small amounts. The tip amounts cluster around the lower end, showing that big tippers are pretty rare.

- Most tips are clustered between \$0 and \$5, with a sharp peak likely around \$2–\$3.
- The distribution is right-skewed, meaning there are fewer occurrences of higher tips.
- A few tips go beyond \$10, but they are very rare (outliers).
- Data cleaning likely removed extreme outliers or zero tips due to data errors.

```
# Histogram AFTER cleaning
ggplot(tripdata_df_clean, aes(x = trip_distance)) +
  geom_histogram(binwidth = 2, fill = "mediumpurple", color = "white") +
  labs(title = "Trip Distance Distribution (After Cleaning)",
       x = "Trip Distance", y = "Count") +
  theme_bw()
```

Trip Distance Distribution (After Cleaning)



Trip Distance Histogram (After Cleaning):

This chart shows how far people usually travel in their rides after cleaning out extreme values. Most trips are on the shorter side, with fewer people taking longer rides—makes sense in a busy city where short commutes are common!

- Most trips are short, typically under 5 units in distance.
- There is a peak around 1–2 units.
- Similar to tip amount, this is also right-skewed with some longer trips, but they happen much less often.
- The longest trips in this cleaned dataset go up to around 30 units, but these are rare.

Question 3 — (30 points): MODELING

```
# Load required package
library(FNN) # For knn.reg()
```

```
## Warning: package 'FNN' was built under R version 4.4.3
```

```
# Define the k-NN regression prediction function
knn.predict <- function(data_train, data_test, k) {
  # Ensure tip_amount is numeric
  data_train$tip_amount <- as.numeric(data_train$tip_amount)
```

```

data_test$tip_amount <- as.numeric(data_test$tip_amount)

# ---- 1. Separate predictors and target ----
x_train <- data_train %>%
  select(-tip_amount) %>%
  select(where(is.numeric)) # keep only numeric predictors

y_train <- data_train$tip_amount

x_test <- data_test %>%
  select(-tip_amount) %>%
  select(where(is.numeric)) # keep only numeric predictors

y_test <- data_test$tip_amount

# ---- 2. Run k-NN regression ----
knn_model <- knn.reg(train = x_train, test = x_test, y = y_train, k = k)

# ---- 3. Extract predictions and calculate MSE ----
predictions <- knn_model$pred
mse <- mean((predictions - y_test)^2)

# ---- 4. Return MSE ----
return(mse)
}

# Run k-NN with k = 5
mse_k5 <- knn.predict(data_train, data_test, k = 5)
head(mse_k5)

```

```
## [1] 1.796742
```

The Mean Squared Error (MSE) of the k-NN regression model with $k = 5$ is approximately 1.80. This value represents the average squared difference between the actual and predicted tip amounts. While an MSE of 1.80 indicates that predictions are generally close to actual values, it also suggests that on average, there can be a noticeable deviation — particularly in higher tip amounts as seen in the scatterplot below.

```

# Prepare the data exactly as in knn.predict() to extract the prediction values alone
x_train <- data_train %>%
  select(-tip_amount) %>%
  select(where(is.numeric))
y_train <- data_train$tip_amount

x_test <- data_test %>%
  select(-tip_amount) %>%
  select(where(is.numeric))
y_test <- data_test$tip_amount

# Run k-NN regression manually
k_value <- 5
knn_model <- knn.reg(train = x_train, test = x_test, y = y_train, k = k_value)

```

```
# Extract predictions
predictions <- knn_model$pred
predictions
```

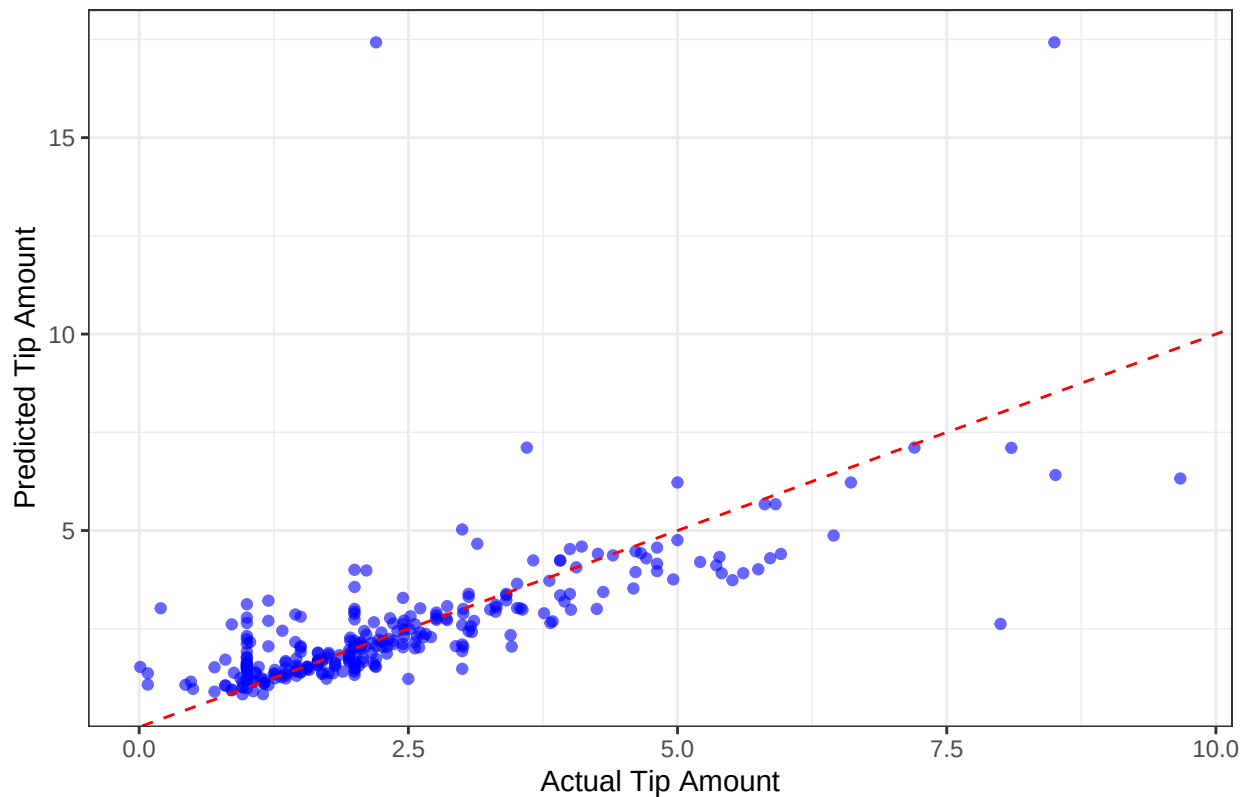
```
## [1] 1.468 2.846 2.446 1.546 7.104 2.740 1.408 7.110 1.886 1.096
## [11] 1.600 1.574 6.414 4.156 2.066 3.216 2.348 1.678 2.194 4.664
## [21] 2.880 2.984 2.300 2.046 2.404 1.080 1.420 1.682 2.054 1.528
## [31] 1.828 2.710 1.660 17.426 6.224 2.040 1.502 1.230 4.566 1.196
## [41] 4.292 4.472 1.348 2.440 1.456 3.526 2.156 1.368 1.014 2.170
## [51] 3.722 1.130 3.076 1.508 3.026 1.504 2.988 2.058 2.054 1.382
## [61] 1.520 1.658 2.114 1.662 1.238 2.338 1.634 3.108 2.024 2.758
## [71] 1.440 1.778 3.024 5.028 2.062 1.660 1.714 1.400 1.566 6.224
## [81] 4.408 1.224 4.404 3.756 1.154 2.440 1.388 2.220 2.284 2.612
## [91] 1.676 2.108 1.890 0.834 2.004 1.382 1.186 0.970 4.000 1.546
## [101] 1.658 1.272 0.952 2.316 1.828 2.768 1.742 1.056 2.062 3.436
## [111] 1.594 2.896 2.864 1.736 4.064 1.520 1.586 2.492 1.808 1.408
## [121] 0.832 2.480 3.350 1.766 1.520 1.500 1.386 2.058 3.224 5.670
## [131] 3.124 1.886 2.986 1.908 4.240 4.238 2.666 1.962 3.388 2.196
## [141] 1.344 1.074 1.376 2.162 0.934 1.556 2.634 3.006 1.366 2.288
## [151] 1.474 1.556 1.752 1.244 2.316 7.110 1.568 1.606 3.736 2.242
## [161] 3.966 3.914 2.234 4.114 4.328 2.162 1.600 1.014 1.540 2.274
## [171] 1.930 4.424 2.648 3.194 2.176 1.828 2.450 2.614 1.714 1.246
## [181] 3.022 1.380 4.592 1.144 2.036 3.006 1.660 2.688 1.056 1.438
## [191] 1.868 1.364 3.312 4.756 4.368 2.818 2.596 4.014 1.506 4.200
## [201] 1.316 1.738 2.888 0.900 1.520 2.722 4.530 1.916 2.042 1.144
## [211] 1.326 2.140 2.700 2.620 3.942 1.878 1.480 1.454 2.408 2.808
## [221] 2.196 1.130 4.872 1.706 1.844 3.050 2.648 1.768 3.020 3.392
## [231] 3.352 4.238 1.898 4.296 2.022 6.324 2.728 2.934 5.670 2.100
## [241] 2.914 17.426 1.064 2.624 2.702 3.984 2.156 3.564 3.646 1.576
## [251] 2.784 3.286 2.008 2.562 2.040 1.206 2.046 2.370 1.588 2.934
## [261] 1.618 3.388 1.658 2.418 1.360 1.528 1.304 1.642 1.224 3.914
## [271] 1.984 0.912 1.382 2.030 0.982 3.006 2.138 1.236 1.532 2.764
## [281] 2.118
```

```
results_df <- data.frame(
  Actual = y_test,
  Predicted = predictions,
  Residuals = predictions - y_test
)
```

```
library(ggplot2)
```

```
ggplot(results_df, aes(x = Actual, y = Predicted)) +
  geom_point(color = "blue", alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  labs(
    title = paste("Actual vs Predicted Tip Amounts (k =", k_value, ")"),
    x = "Actual Tip Amount",
    y = "Predicted Tip Amount"
  ) +
  theme_bw()
```

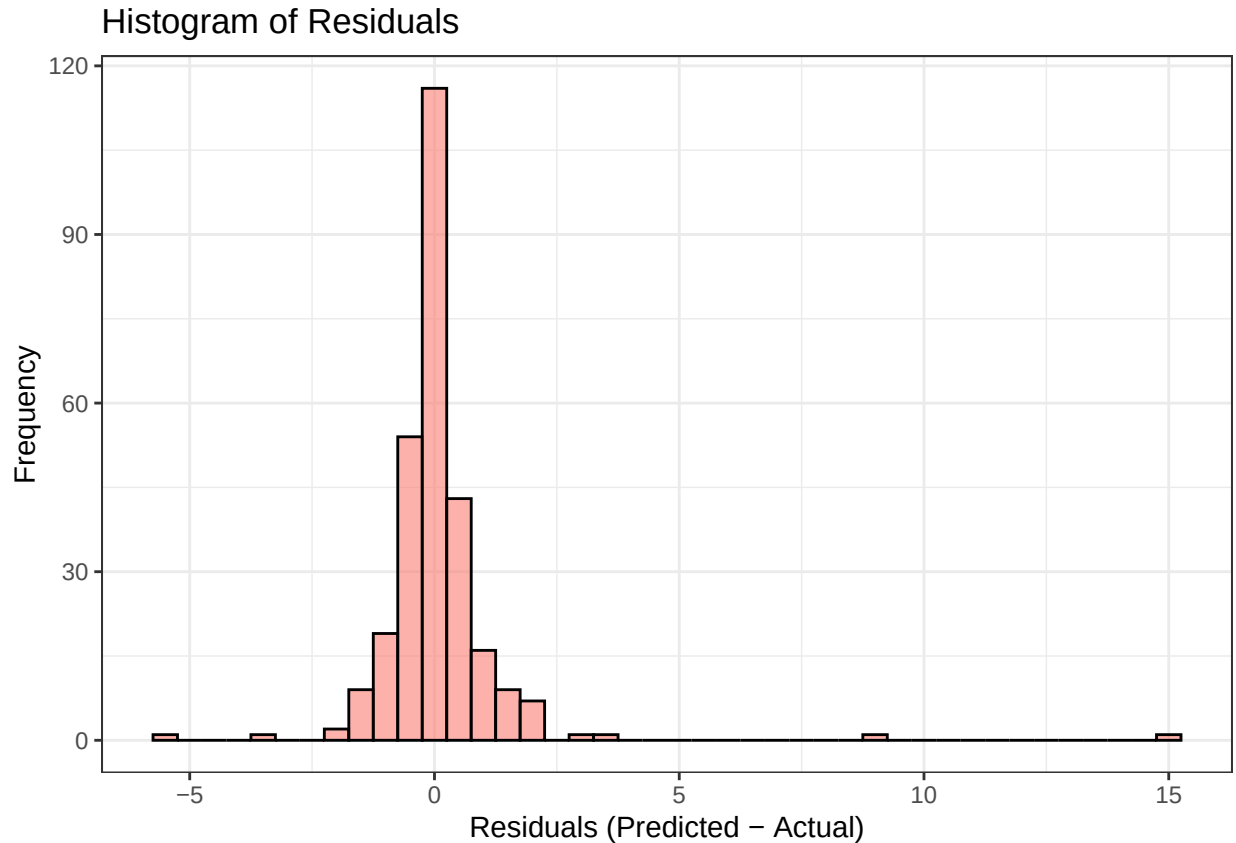
Actual vs Predicted Tip Amounts (k = 5)



The “Actual vs Predicted Tip Amounts” plot with $k = 5$ from the k-NN regression model shows how well the model is performing.

The scatterplot comparing actual vs. predicted tip amounts for $k = 5$ provides valuable insight into the performance of our k-NN regression model. The majority of data points cluster closely around the diagonal reference line, suggesting that the model performs well for predicting lower to moderate tip amounts, which constitute most of the dataset. However, as tip amounts increase, the prediction error becomes more noticeable, with several outliers deviating significantly from the expected trend. These discrepancies are likely due to the sparsity of high-tip data points, which poses a challenge for instance-based models like k-NN. Overall, while the model demonstrates acceptable accuracy for common scenarios, its performance diminishes for less frequent, higher-tip cases. Further tuning of the k value and incorporating additional feature engineering or outlier handling could potentially enhance predictive accuracy.

```
# Plot Histogram of Residuals
ggplot(results_df, aes(x = Residuals)) +
  geom_histogram(binwidth = 0.5, fill = "salmon", color = "black", alpha = 0.6) +
  labs(
    title = "Histogram of Residuals",
    x = "Residuals (Predicted - Actual)",
    y = "Frequency"
  ) +
  theme_bw()
```

Residuals

Residuals are the differences between the actual values and the predicted values produced by a model. In simpler terms, they represent the error or prediction error of the model for each data point. Residuals help us understand how well a model fits the data.

Key Observations:

- **Peak Around Zero:** The histogram has a sharp peak near 0 on the x-axis, suggesting that most of the predictions are very close to the actual values.
- **Spread of Residuals:** The spread of residuals on either side of zero is not too large. The histogram tailing off towards the positive and negative values implies that there are some instances where the model's predictions are off by a larger amount, but these are not as frequent.

Interpretation:

- **Good Model Fit:** The fact that the residuals are clustered around zero suggests that the k-NN regression model is performing well for the majority of the data, with a small number of outliers.
- **Small Variance:** The distribution appears to be relatively narrow, indicating that the model is consistent in its predictions for most of the data.
- **Potential Outliers:** The presence of some large residuals may suggest that there are certain instances where the model struggles to predict the tip amount accurately, possibly due to unusual or rare data points.

Question 4 — (30 points): EVALUATION

```
# Define a range of k values from 1 to 40 (or 1 to 20 if strictly 20 values required)
k_values <- 1:20

# Empty vector to store MSE for each k
mse_values <- numeric(length(k_values))

# Loop through each k, call knn.predict(), and store the MSE
for (i in seq_along(k_values)) {
  mse_values[i] <- knn.predict(data_train, data_test, k = k_values[i])
}

# Combine into a dataframe for plotting
mse_df <- data.frame(
  k = k_values,
  MSE = mse_values
)

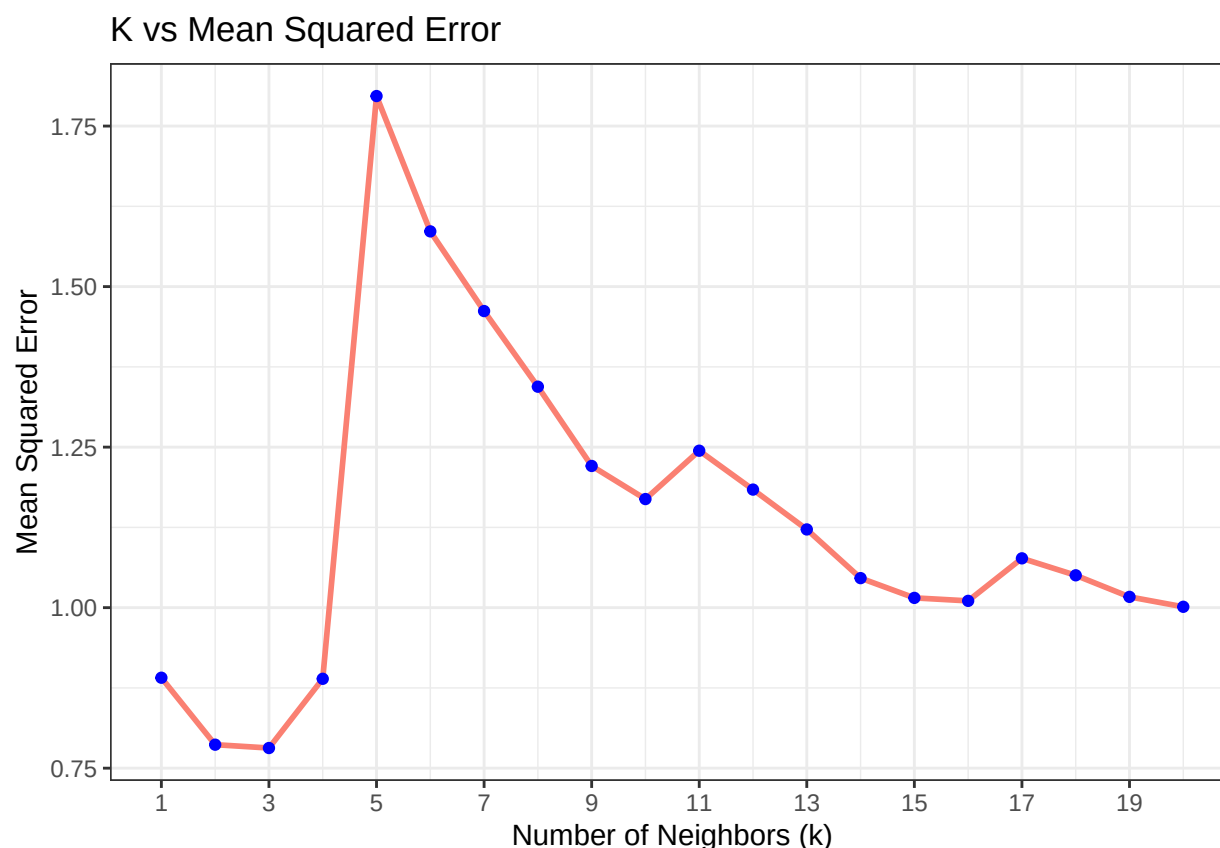
mse_df
```

```
##      k      MSE
## 1     1 0.8908452
## 2     2 0.7866006
## 3     3 0.7813924
## 4     4 0.8892060
## 5     5 1.7967422
## 6     6 1.5859693
## 7     7 1.4618949
## 8     8 1.3441313
## 9     9 1.2206550
## 10    10 1.1691090
## 11    11 1.2444225
## 12    12 1.1838513
## 13    13 1.1218168
## 14    14 1.0459941
## 15    15 1.0152076
## 16    16 1.0105655
## 17    17 1.0767974
## 18    18 1.0503616
## 19    19 1.0167700
## 20    20 1.0013143
```

```
# Plot the MSE vs k values
library(ggplot2)

ggplot(mse_df, aes(x = k, y = MSE)) +
  geom_line(color = "salmon", size = 1) +
  geom_point(color = "blue") +
  scale_x_continuous(breaks = seq(min(mse_df$k), max(mse_df$k), by = 2)) +
  labs(
    title = "K vs Mean Squared Error",
    x = "Number of Neighbors (k)",
```

```
y = "Mean Squared Error"
) +
theme_bw()
```



Explanation of the Chart and Best Value of k:

The chart displays the relationship between the number of neighbors (k) and the corresponding mean squared error (MSE) in the K-Nearest Neighbors model. Initially, the MSE drops sharply from $k = 1$ to $k = 3$, where it reaches the lowest point. After this, there is a significant spike in error at $k = 5$, followed by a gradual decrease and fluctuations as k increases. Based on this trend, $k = 3$ is the most suitable choice because it yields the lowest MSE, indicating the most accurate prediction performance for the given dataset. Choosing a higher k smooths predictions but can increase bias, while too small a k risks overfitting. Therefore, $k = 3$ offers the best balance in this case.

Thoughts on the model that we developed and the accuracy of its predictions and advocate for its use to predict the tip amount of future trips:

Based on the results and the MSE trend across different values of k , the K-Nearest Neighbors (KNN) model demonstrates reasonable predictive performance. The lowest MSE is achieved at $k = 3$ with a value of approximately 0.781, suggesting that the model performs best at this setting. However, we observe significant variability in error values, especially a spike at $k = 5$, indicating sensitivity to parameter choice. While the overall MSE values remain moderate beyond $k = 3$, they do not significantly outperform the minimum. This shows that the model captures patterns in the data but may not generalize well under all conditions due to its non-parametric nature and reliance on distance metrics. Given these observations, I would recommend cautious use of the KNN model for predicting tip amounts in future trips. It may serve well in situations with similar data characteristics but might not perform reliably on unseen or differently distributed datasets. Further validation and comparison with alternative models like decision trees or linear regression could strengthen confidence in its predictive capabilities.

Question 5 — (10 optional/ bonus points)

```
# Custom Bright Pastel Colors
bright_pastel_colors <- c(
  "Bronx" = "#77DD77",
  "Brooklyn" = "#779ECB",
  "Manhattan" = "#FFD580",
  "Queens" = "#FF6961"
)

ggplot(tripdata_df_clean, aes(x = Borough_pickup)) +

  geom_bar(aes(fill = Borough_pickup), show.legend = FALSE, width = 0.6) +

  scale_fill_manual(values = bright_pastel_colors) +

  # Micro-insights for each Borough
  annotate("text", x = 1, y = 50, label = "Lowest Taxi Usage", size = 4, fontface = "bold", color = "#1f77b4"),
  annotate("text", x = 2, y = 500, label = "Residential + Local Commutes", size = 4, fontface = "bold", color = "#1f77b4"),
  annotate("text", x = 3, y = 650, label = "Business, Tourists & Nightlife", size = 4, fontface = "bold", color = "#1f77b4"),
  annotate("text", x = 4, y = 400, label = "Airport Driven Demand", size = 4, fontface = "bold", color = "#1f77b4")

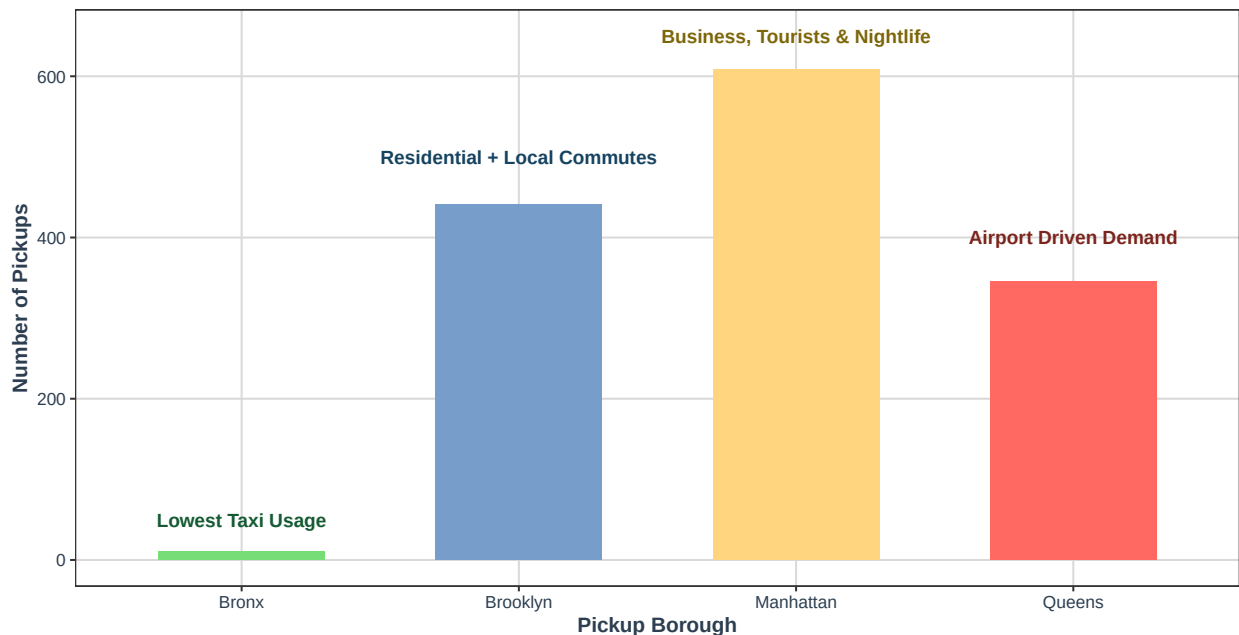
  # Main Title & Subtitle
  labs(
    title = "NYC Taxi Ride Culture: Where City Meets Speed & Demand",
    subtitle = "Taxi demand mirrors urban lifestyle - Manhattan dominates for its business, tourism, and transit",
    x = "Pickup Borough",
    y = "Number of Pickups",
    caption = "Source: NYC Green Taxi Trip Data"
  ) +

  theme_bw(base_size = 12) +

  theme(
    plot.title = element_text(face = "bold", size = 18, color = "#2C3E50"),
    plot.subtitle = element_text(size = 12, color = "#7F8C8D", margin = margin(b = 10)),
    axis.title = element_text(face = "bold", color = "#2C3E50"),
    axis.text = element_text(color = "#2C3E50"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank(),
    plot.caption = element_text(size = 9, color = "gray50", hjust = 1)
  )
```

NYC Taxi Ride Culture: Where City Meets Speed & Demand

Taxi demand mirrors urban lifestyle — Manhattan dominates for its business, tourism, and 24x7 movement



Source: NYC Green Taxi Trip Data

Taxi Usage Across NYC Boroughs:

The chart highlights the varying taxi demand across NYC's boroughs, showing how each borough has different usage patterns based on local needs:

Manhattan leads with the highest number of taxi pickups, driven by business, tourism, and nightlife. This reflects Manhattan's role as the city's economic and cultural hub, where taxis serve both local residents and visitors.

Brooklyn shows significant taxi usage primarily for residential and local commuting. People in Brooklyn use taxis for their day-to-day needs, such as going to work or running errands, making it the second most active borough for taxi demand.

Queens sees taxi demand driven largely by airport travel, with people needing rides to and from JFK and LaGuardia airports. This highlights the borough's critical role in air travel, where taxis are a convenient way to get to and from the airports.

The Bronx has the lowest taxi usage, which suggests that people in the Bronx may rely more on public transportation or personal vehicles, as taxis are not the primary mode of transport.

What This Tells Us About NYC's Taxi Culture:

Manhattan's Taxi Dependence: The dominance of Manhattan in taxi usage highlights its pivotal role in the city's economy, culture, and tourism. Taxis here cater to both everyday needs and visitors from all around the world, making them a crucial part of NYC's transportation ecosystem.

Local vs. Tourist Demand: Boroughs like Brooklyn and Queens have a more localized demand. While Brooklyn taxis are primarily used for commuting and residential needs, Queens caters significantly to tourists and travelers heading to the airports. These boroughs show how taxis are tailored to serve different local lifestyles.

Public vs. Private Transportation: The contrast between the Bronx and other boroughs could suggest a preference for alternative transportation options in less central areas, where taxis aren't the primary mode of travel.

```
library(ggtext)
```

```
## Warning: package 'ggtext' was built under R version 4.4.3
```

```
# Your Data
```

```
mse_split_df <- data.frame(  
  Split_Percentage = c(0.5, 0.6, 0.7, 0.8, 0.9),  
  MSE = c(3.19, 3.3, 0.88, 0.78, 1.00)  
)
```

```
ggplot(mse_split_df, aes(x = Split_Percentage, y = MSE)) +
```

```
# Background Shading
```

```
annotate("rect", xmin = 0.5, xmax = 0.65, ymin = -Inf, ymax = Inf, alpha = 0.1, fill = "#E74C3C") +  
annotate("rect", xmin = 0.66, xmax = 0.85, ymin = -Inf, ymax = Inf, alpha = 0.1, fill = "#27AE60") +
```

```
# Line & Points
```

```
geom_line(color = "#2980B9", size = 1.5) +  
geom_point(size = 5, shape = 21, fill = "#1ABC9C", color = "black") +  
geom_text(aes(label = round(MSE, 2)), vjust = -1.2, size = 4, fontface = "bold", color = "#2C3E50") +
```

```
# Labels for Zones
```

```
annotate("text", x = 0.56, y = 3.5, label = "Undertrained", color = "#C0392B", size = 4, fontface = "bold", align = "center") +  
annotate("text", x = 0.75, y = 2.5, label = "Optimal Zone", color = "#145A32", size = 4, fontface = "bold", align = "center") +  
annotate("text", x = 0.87, y = 1.6, label = "Slight Overfitting →", color = "#CA6F1E", size = 4.2, fontface = "bold", align = "center") +
```

```
# Labels
```

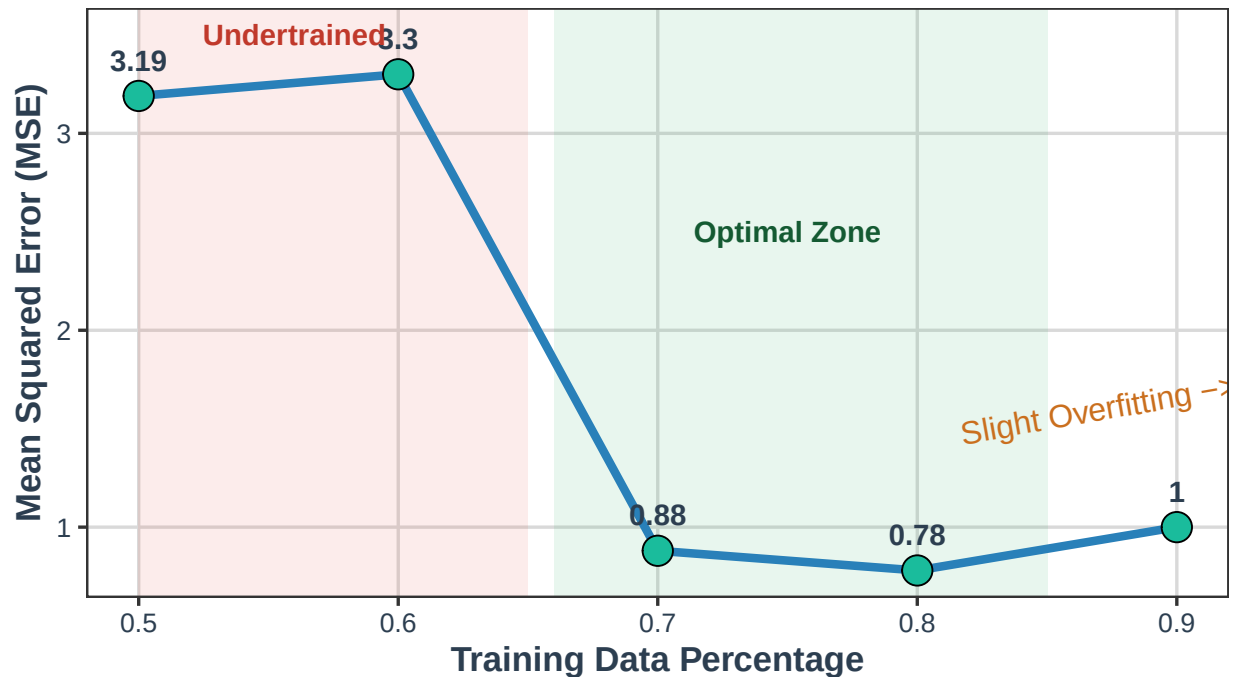
```
labs(  
  title = "Sweet Spot in Learning: Train/Test Split vs MSE",  
  subtitle = "KNN performs best when trained on 70-80% of data - too little leads to underfitting",  
  x = "Training Data Percentage",  
  y = "Mean Squared Error (MSE)",  
  caption = "Source: NYC Green Taxi Trip Data | Model: k-Nearest Neighbors"  
) +
```

```
theme_bw(base_family = "sans", base_size = 13) +
```

```
theme(  
  plot.title = element_text(face = "bold", size = 18, color = "#2C3E50"),  
  plot.subtitle = element_text(size = 12, color = "#7F8C8D", margin = margin(b = 10)),  
  axis.title = element_text(face = "bold", color = "#2C3E50"),  
  panel.grid.major = element_line(color = "gray85"),  
  panel.grid.minor = element_blank(),  
  plot.caption = element_text(size = 9, hjust = 1, color = "gray40"),  
  axis.text = element_text(color = "#2C3E50")  
)
```

Sweet Spot in Learning: Train/Test Split vs MSE

KNN performs best when trained on 70–80% of data — too little leads to underfitting



Source: NYC Green Taxi Trip Data | Model: k-Nearest Neighbors

The graph explores the relationship between the percentage of data used for training (versus testing) and the model's Mean Squared Error (MSE) in the context of the k-Nearest Neighbors (k-NN) regression model. The goal is to find the optimal train/test split that minimizes prediction error while avoiding overfitting or underfitting.

1. Undertraining Zone:

The graph shows that when less than 70% of the data is used for training, the model's MSE remains high (around 3.19).

This indicates underfitting, where the model is not able to learn enough from the data to make accurate predictions. When too little data is used for training, the model doesn't have enough information to capture the underlying patterns of the dataset.

Effect of undertraining: The model performs poorly because it doesn't have sufficient data to learn complex relationships, resulting in high MSE.

2. Optimal Zone:

As we increase the percentage of training data to around 70-80%, we enter the optimal zone where the MSE decreases significantly, reaching its lowest point at 0.78.

In this zone, the model is able to learn enough from the data without being overwhelmed by too much noise or overfitting. The model's predictions improve, as it finds the right balance between learning from data and not overfitting to it.

Effect of optimal training: With a 70-80% train/test split, the k-NN model is at its best, providing accurate and reliable predictions, which helps make the model more generalized and robust.

3. Overfitting Zone:

As we approach 90% or more of the data used for training, the model's MSE starts to increase again to 0.88.

This is indicative of slight overfitting. Here, the model has learned the training data too well, even capturing the noise and outliers. This causes it to perform less well on the test set since it has become too specialized to the training data.

Effect of overfitting: While more data can theoretically improve the model's performance, too much training data relative to testing data can make the model too sensitive to the training set, thus diminishing its ability to generalize effectively.